

数据结构与算法

排序

排序

- 9. 1 概述
- 9. 2 插入排序
- 9. 3 交换排序
- 9. 4 选择排序
- 9. 5 归并排序
- 9. 6 基数排序
- 9. 7 各种内部排序方法讨论
- 9. 8 外部排序

9.1 概述

- 排序是计算机内经常进行的一种操作，其目的是将一组“无序”的数据元素（简称为元素，也可称为记录）序列调整为“有序”的元素序列。
- 排序：将数据元素（或记录）的任意序列，按照关键字有序的序列重新排列。

9.1 概述

- 关键字：是数据元素中的某个数据项。如果某个数据项可以唯一地确定一个数据元素，就将其称为主关键字；否则，称为次关键字。
- 排序算法的稳定性：如果在待排序的记录序列中有多个数据元素的关键字值相同，经过排序后，这些数据元素的相对次序保持不变，则称这种排序算法是稳定的，否则称之为不稳定的。

9.1 概述

- 内部排序：待排序的数据元素全部存入计算机的内存中，在排序中不需要访问外存
- 外部排序：待排序的数据元素不能全部装入内存，在排序过程中需要不断访问外存

9.1 概述

- 基于不同的“扩大”有序序列长度的方法，内部排序方法大致可分下列几种类型：插入、交换、选择、归并和基数。
- 在排序过程中，一般进行两种基本操作：比较两个关键字的大小；将记录从一个位置移动到另一个位置。

9.1 概述

- 插入类：将无序子序列中的一个元素“插入”到有序序列中，从而增加元素的有序子序列的长度。
- 交换类：通过“交换”无序序列中的元素从而得到其中关键字最小或最大的元素，并将它加入到有序子序列中，以此方法增加元素的有序子序列的长度。

9.1 概述

- 选择类：从元素的无序子序列中“选择”关键字最小或最大的元素，并将它加入到有序子序列中，以此方法增加元素的有序子序列的长度。
- 归并类：通过“归并”两个(2路归并)、三个(3路归并)或多个(多路归并)有序子序列，逐步增加有序序列的长度。

9.1 概述

排序算法的效率，评价排序算法的效率主要有两点

- 一是在数据量规模一定的条件下，算法执行所消耗的平均时间，对于排序操作，时间主要消耗在关键字之间的比较和数据元素的移动上，因此我们可以认为高效率的排序算法应该是尽可能少的比较次数和尽可能少的数据元素移动次数；

- 二是执行算法所需要的辅助存储空间，辅助存储空间是指在数据量规模一定的条件下，除了存放待排序数据元素占用的存储空间之外，执行算法所需要的其他存储空间，理想的空间效率是算法执行期间所需要的辅助空间与待排序的数据量无关。

9.2 插入排序

- 插入排序的基本思想是：在一个已排好序的记录子集的基础上，每一步将下一个待排序的记录有序地插入到已排好序的记录子集中，直到将所有待排记录全部插入为止。
- 打扑克牌时的抓牌就是插入排序一个很好的例子，每抓一张牌，插入到合适位置，直到抓完牌为止，即可得到一个有序序列。

9.2.1 直接插入排序

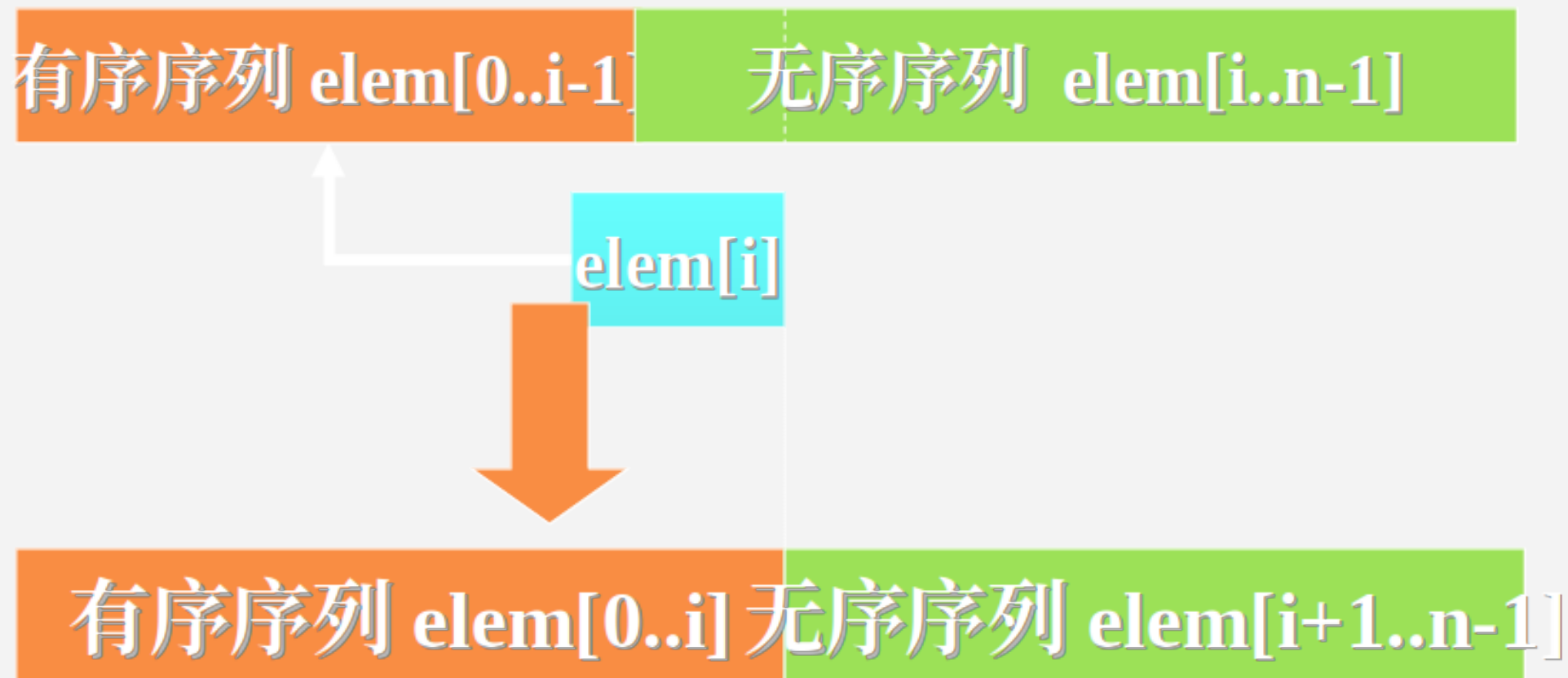
- 直接插入排序是一种最基本的插入排序方法。其基本操作是将第 i 个记录插入到前面 $i-1$ 个已排好序的记录中。
- 完整的直接插入排序是从 $i=2$ 开始的，也就是说，将第1个记录视为已排好序的单元元素子集合，然后将第2个记录插入到单元元素子集合中。 i 从2循环到 n ，即可实现完整的直接插入排序。

9.2.1 直接插入排序

A)	{ 48 }	62	35	77	55	14	<u>35</u>	98
B)	{ 48	62 }	35	77	55	14	<u>35</u>	98
C)	{ 35	48	62 }	77	55	14	<u>35</u>	98
D)	{ 35	48	62	77 }	55	14	<u>35</u>	98
E)	{ 35	48	55	62	77 }	14	<u>35</u>	98
F)	{ 14	35	48	55	62	77 }	<u>35</u>	98
G)	{ 14	35	<u>35</u>	48	55	62	77 }	98
H)	{ 14	35	<u>35</u>	48	55	62	77	98 }

9.2.1 直接插入排序

- 一趟直接插入排序的基本思想



9.2.1 直接插入排序

实现“一趟插入排序”的步骤

- 在 $\text{elem}[0..i-1]$ 中查找 $\text{elem}[i]$ 的插入位置,
 $\text{elem}[0..j] \leq \text{elem}[i] < \text{elem}[j+1..i-1]$
- 将 $\text{elem}[j+1..i-1]$ 中的所有元素均后移一个位置
- 将 $\text{elem}[i]$ 插入 (复制) 到 $\text{elem}[j+1]$ 的位置上

9.2.1 直接插入排序

利用 “顺序查找” 实现插入排序的要点：

- 从 $elem[i-1]$ 起向前进行顺序查找
- 对于在查找过程中找到的那些关键字大于 $elem[i]$ ($=e$) 的关键字的元素，在查找的同时实现元素向后移动

9.2.1 直接插入排序

- 从时间耗费角度来看，主要时间耗费在关键字比较和移动元素上。
- 直接插入排序的时间复杂度为 $T(n)=O(n^2)$ 。
- 直接插入排序算法简便，比较适用于待排序记录数较少且基本有序的情况。在直接插入排序法的基础上，从减少“比较关键字”和“移动记录”两种操作的次数着手来进行改进。

9.2.2 希尔（Shell）排序

- 希尔排序方法又称为缩小增量排序，其基本思想是将待排序的记录划分成几组，从而减少参与直接插入排序的数据量，当经过几次分组排序后，记录的排列已经基本有序，这个时候再对所有的记录实施直接插入排序。

9.2.2 希尔 (Shell) 排序

具体步骤可以描述如下：

1、假设待排序的记录为 n 个，先取整数 $d < n$ ， d 称为增量。例如，取 $d = n/2$ （ $n/2$ 表示不大于 $n/2$ 的最大整数），将所有距离为 d 的记录构成一组，从而将整个待排序记录序列分割成为 d 个子序列，对每个分组分别进行直接插入排序。

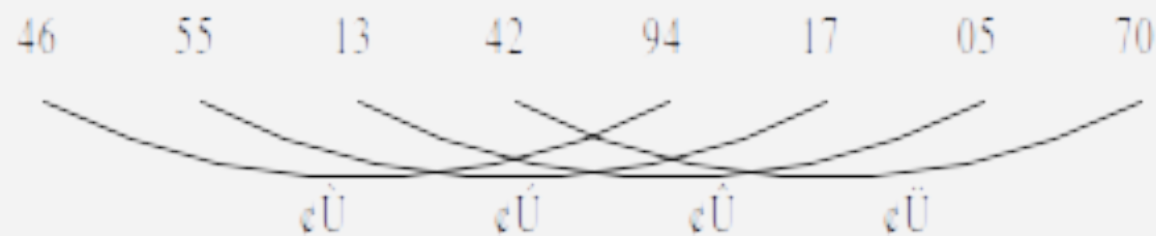
9.2.2 希尔 (Shell) 排序

具体步骤可以描述如下：

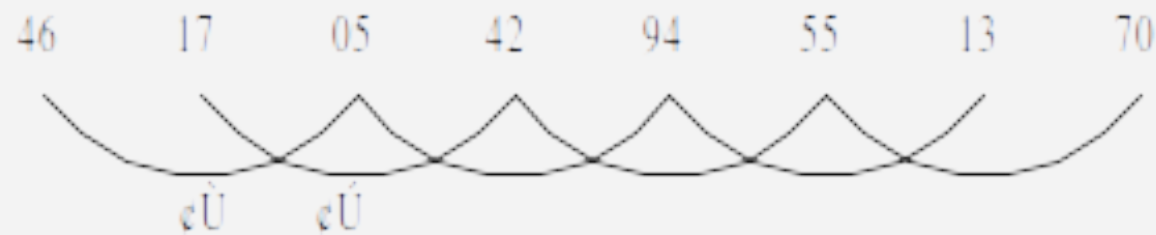
2、然后再缩小间隔 d ，例如，取 $d=d/2$ ，重复上述的分组，再对每个分组分别进行直接插入排序，直到最后取 $d=1$ ，即将所有记录放在一组进行一次直接插入排序，最终将所有记录重新排列成按关键字有序的序列。

9.2.2 希尔 (Shell) 排序

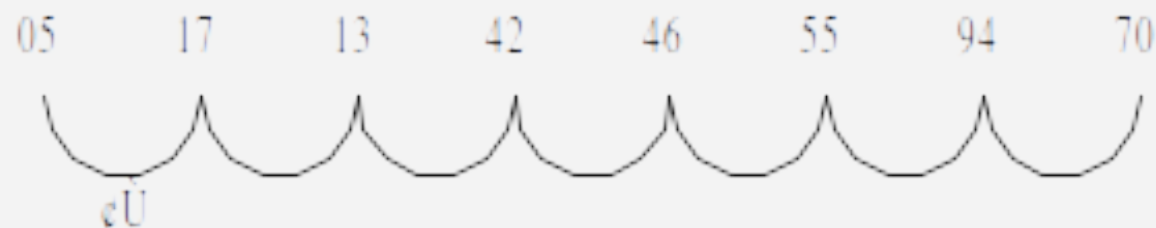
初始关键字序列:



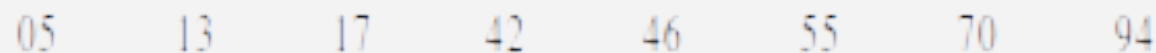
取 $d_1 \in \frac{1}{2}4$, 分为 4 个间隔为 4 的子序列,
各子序列内进行插入排序, 结果为:



取 $d_2 \in \frac{1}{2}2$, 分为 2 个间隔为 2 的子序列,
各子序列内进行插入排序, 结果为:



取 $d_3 \in \frac{1}{2}1$, 分为 1 个间隔为 1 的子序列,
最后排序结果为:



9.2.2 希尔 (Shell) 排序

16	25	12	30	47	11	23	36	9	18	31
----	----	----	----	----	----	----	----	---	----	----

第一趟希尔排序，设增量 $d=5$

11	23	12	9	18	16	25	36	30	47	31
----	----	----	---	----	----	----	----	----	----	----

第二趟希尔排序，设增量 $d=3$

9	18	12	11	23	16	25	31	30	47	36
---	----	----	----	----	----	----	----	----	----	----

第三趟希尔排序，设增量 $d=1$

9	11	12	16	18	23	25	30	31	36	47
---	----	----	----	----	----	----	----	----	----	----

9.2.2 希尔 (Shell) 排序

- 希尔排序是一个较好的插入排序方法。希尔排序能迅速减少逆转数，尽管当间隔为1时，希尔排序相当于直接插入排序，但此时的关键字序列的逆转数已经很小，序列已经基本有序，使用的恰好是直接插入的最佳性质。
- 希尔排序的分析是一个复杂的问题，因为它的时空耗费是所取的“增量”序列的函数。到目前为止，尚未有人求得一种最好的增量序列。但经过大量研究，也得出了一些局部的结论。

9.3 交换排序

- 这是一类借助交换进行排序的方法。
- 其中最简单的一种是冒泡排序，最先进的一种是快速排序。

9.3.1 冒泡排序

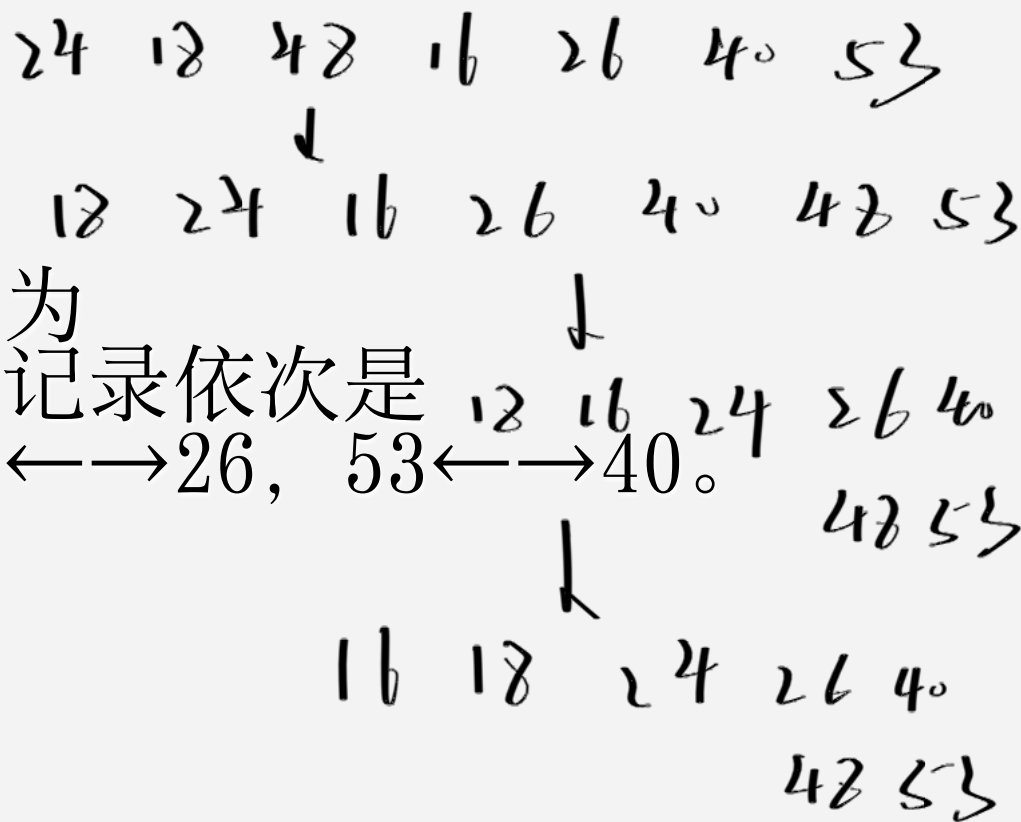
- 冒泡排序是交换排序中一种简单的排序方法。它的基本思想是对所有相邻记录的关键字值进行比较，如果是逆序 ($a[j] > a[j+1]$)，则将其交换，最终达到有序化。
- 冒泡排序的最好、最坏和平均情况下的时间复杂度是相同的，都是 $O(n^2)$ 。

9.3.1 冒泡排序

一组记录的排序码为(48, 24, 18, 53, 16, 26, 40)，采用冒泡排序法进行排序，则第一趟排序需要进行记录交换的次数是 (C)。

A) 3 B) 4 C) 5 D) 6

【分析】经过一趟冒泡排序后的序列为
{48, 24, 18, 53, 16, 26, 40}，交换的记录依次是
48 $\longleftrightarrow$ 24, 48 $\longleftrightarrow$ 18, 53 $\longleftrightarrow$ 16, 53 $\longleftrightarrow$ 26, 53 $\longleftrightarrow$ 40。
【答案：C】



9.3.2 快速排序

- 快速排序：是对冒泡排序的一种改进。它的基本思想是，通过一趟排序将待排记录分割成独立的两部分，其中一部分记录的关键字均比另一部分记录的关键字小，则可分别对这两部分记录继续进行排序，以达到整个序列有序。

9.3.2 快速排序

快速排序的步骤

步骤 1: 初始化指针与枢轴, 选数组第一个元素作为枢轴 `pivot`; 左指针 `i` 从左端开始, 右指针 `j` 从右端开始。

步骤 2: 双指针相向扫描+交换:

先移动右指针 `j`: 向左找第一个小于 `pivot` 的元素;

再移动左指针 `i`: 向右找第一个大于 `pivot` 的元素;

若 $i < j$: 交换它们, 然后继续移动指针;

当 $i \geq j$ 时停止扫描: 此时 `j` 是枢轴的最终位置, 交换枢轴和元素 `j`, 让枢轴归位。

9.3.2 快速排序

快速排序的步骤

步骤 3: 递归处理

枢轴归位后，左区域 $[low, j-1]$ 、右区域 $[j+1, high]$ 分别递归排序。

48	62	35	77	55	14	<u>35</u>	98
----	----	----	----	----	----	-----------	----

i

j

48	<u>35</u>	35	77	55	14	62	98
----	-----------	----	----	----	----	----	----

i

j

48	<u>35</u>	35	14	55	77	62	98
----	-----------	----	----	----	----	----	----

i==j

14	<u>35</u>	35	48	55	77	62	98
----	-----------	----	----	----	----	----	----

```
int partition_hoare(int arr[], int low, int high) {  
    // 步骤1: 选第一个元素作为枢轴  
    int pivot = arr[low];  
    int i = low;    // 左指针: 从左端开始  
    int j = high;   // 右指针: 从右端开始  
  
    // 步骤2: 双指针向中间靠拢, 交替比较交换  
    while (i < j) {  
        // 右指针向左找: 第一个小于枢轴的元素 (跳过≥枢轴的)  
        while (i < j && arr[j] >= pivot) {  
            j--;  
        }  
        // 左指针向右找: 第一个大于枢轴的元素 (跳过≤枢轴的)  
        while (i < j && arr[i] <= pivot) {  
            i++;  
        }  
        // 找到后交换左右指针指向的元素  
        if (i < j) {  
            swap(&arr[i], &arr[j]);  
        }  
    }  
  
    // 步骤3: 将枢轴放到最终位置 (i=j的位置)  
    swap(&arr[low], &arr[j]);  
    return j;    // 返回枢轴最终下标  
}
```

```
void quickSort_hoare(int arr[], int low, int high) {  
    if (low < high) {  
        // 一趟快速排序：让枢轴归位，划分左右区域  
        int pivot_pos = partition_hoare(arr, low, high);  
        // 递归处理左区域（枢轴左侧）  
        quickSort_hoare(arr, low, pivot_pos - 1);  
        // 递归处理右区域（枢轴右侧）  
        quickSort_hoare(arr, pivot_pos + 1, high);  
    }  
}
```

Lomuto 分区方案

```
int partition(int arr[], int low, int high) {  
    // 选择数组最后一个元素作为基准值  
    int pivot = arr[high];  
    // i: 小于等于基准值区域的右边界 (初始为low-1)  
    int i = low - 1;  
  
    // 遍历[low, high-1]区间的元素  
    for (int j = low; j < high; j++) {  
        // 若当前元素≤基准值, 将其划入左区域  
        if (arr[j] <= pivot) {  
            i++; // 左区域右边界右移  
            swap(&arr[i], &arr[j]); // 交换元素  
        }  
    }  
  
    // 将基准值放到最终位置 (i+1)  
    swap(&arr[i + 1], &arr[high]);  
    return (i + 1); // 返回基准值下标  
}
```


9.3.2 快速排序

维度	「Hoare 分区方案」（原始版）	「Lomuto 分区方案」（简化版）
枢轴初始选择	通常选第一个元素	通常选最后一个元素
扫描方式	双指针（左 / 右）相向扫描（从两端向中间靠拢）	单指针（左边界标记）单向扫描（从左到右遍历）
交换触发条件	左指针找 $>$ 枢轴的元素、右指针找 $<$ 枢轴的元素，找到后交换两指针元素，最终让枢轴归位	遍历指针找 $\leq$ 枢轴的元素，交换到“左区域”，最后将枢轴放到左区域右侧
交换次数	更少（平均仅 Lomuto 的 $1/3$ ）	更多（但逻辑更简单）
实现复杂度	稍高（需处理指针越界、最终枢轴交换）	更低（新手易理解）

9.3.2 快速排序

- 到目前为止快速排序是在同数量级 $O(n \lg n)$ 的排序算法中，平均性能最好的一种排序方法。
- 但当原始记录排列基本有序或基本逆序时，每一趟的基准记录有可能只将其余记录分成一部分，这样就降低了时间效率，所以快速排序适用于原始记录排列杂乱无章的情况。

9.3.2 快速排序

- 快速排序需一个栈空间来实现递归。空间复杂度为 $O(\lg n)$ 。

9.3.2 快速排序

采用递归方式对顺序表进行快速排序，下列关于递归次数的叙述中，正确的是（D）

- A: 递归次数与初始数据的排列次序无关
- B: 每次划分后，先处理较长的分区可以减少递归次数
- C: 每次划分后，先处理较短的分区可以减少递归次数
- D: 递归次数与每次划分后得到的分区处理顺序无关

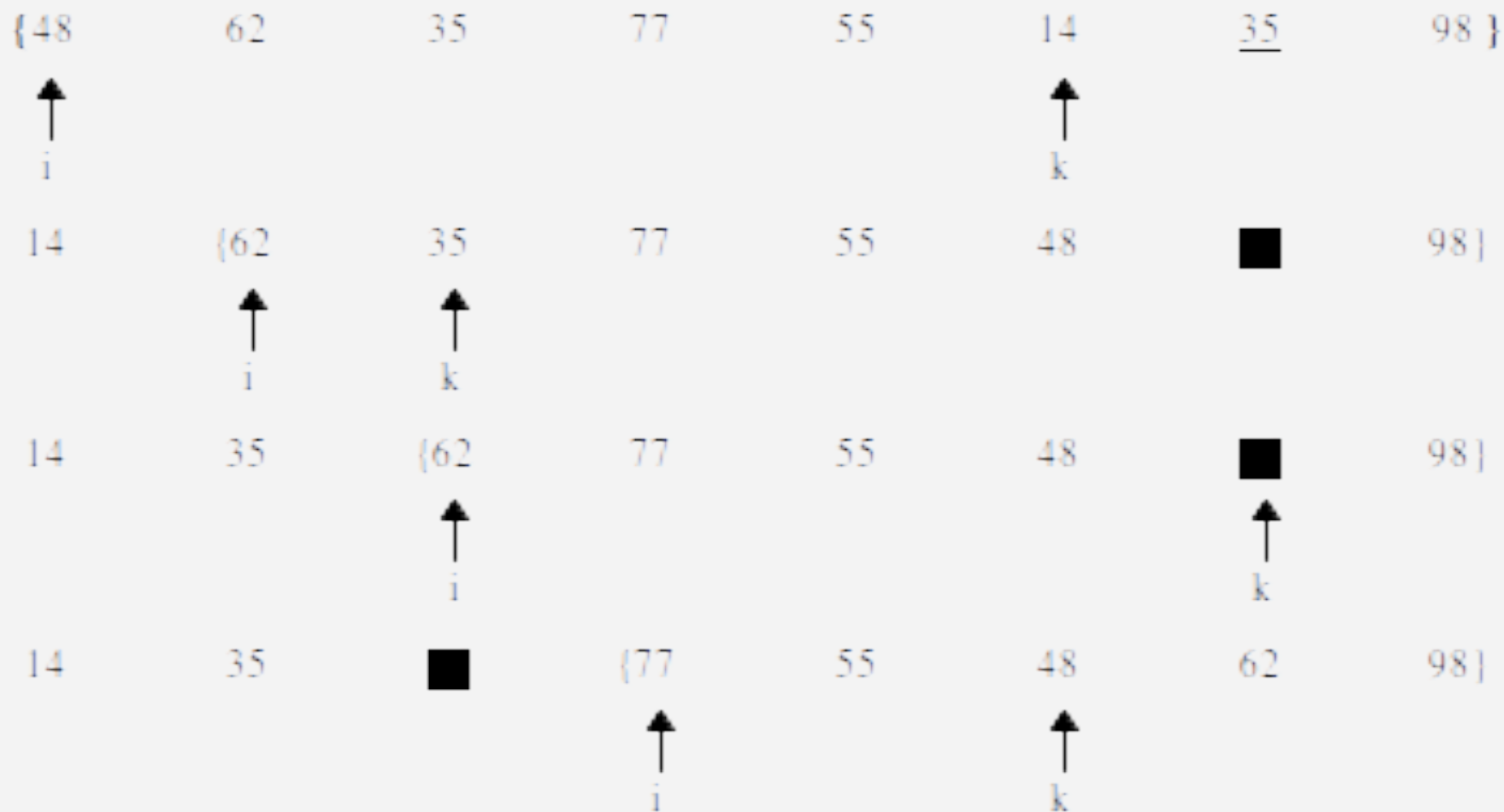
9.4 选择排序

选择排序是指在排序过程中，依次从待排序的记录序列中选出关键字值最小的记录、关键字值次小的记录，并分别将它们定位到序列左侧的第1个位置、第2个位置；最后剩下一个关键字值最大的记录位于序列的最后一个位置，从而使待排序的记录序列成为按关键字值由小到大排列的有序序列。

9.4.1 简单选择排序

- 简单选择排序的基本思想：第 i 趟简单选择排序是指通过 $n-i$ 次关键字的比较，从 $n-i+1$ 个记录中选出关键字最小的记录，并与第 i 个记录进行交换。共需进行 $i-1$ 趟比较，直到所有记录排序完成为止。
- 例如：进行第1趟选择时，从当前候选记录中选出关键字最小的 k 号记录，并与第1个记录进行交换。

9.4.1 简单选择排序



9.4.1 简单选择排序

```
// 选择排序函数 (升序)
void selectionSort(int arr[], int n) {
    int i, j, minIndex, temp;

    // 遍历数组的每一个位置
    for (i = 0; i < n - 1; i++) {
        // 假设当前位置是最小值的位置
        minIndex = i;

        // 在未排序部分中寻找最小元素的索引
        for (j = i + 1; j < n; j++) {
            if (arr[j] < arr[minIndex]) {
                minIndex = j;
            }
        }

        // 如果最小值不在当前位置，则交换
        if (minIndex != i) {
            temp = arr[i];
            arr[i] = arr[minIndex];
            arr[minIndex] = temp;
        }
    }
}
```


9.4.2 堆排序

- 堆排序是另一种基于选择的排序方法。只需要一个元素的辅助存储空间。
- 堆定义：堆是满足下列性质的数列 $\{e_0, e_1, \dots, e_{n-1}\}$ 。

$$\begin{cases} e_i \leq e_{2i+1} \\ e_i \leq e_{2i+2} \end{cases}$$

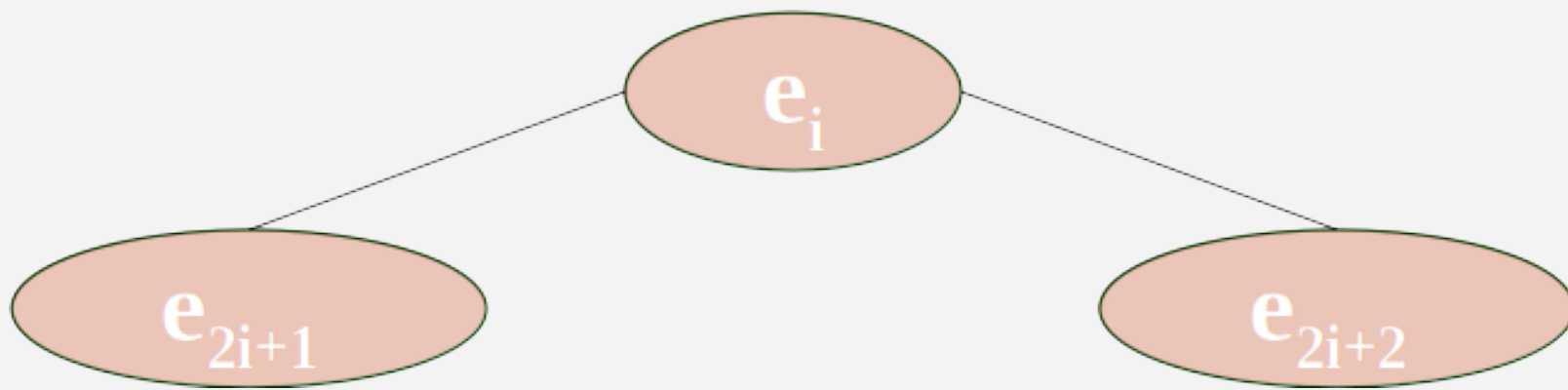
(小顶堆)

$$\begin{cases} e_i \geq e_{2i+1} \\ e_i \geq e_{2i+2} \end{cases}$$

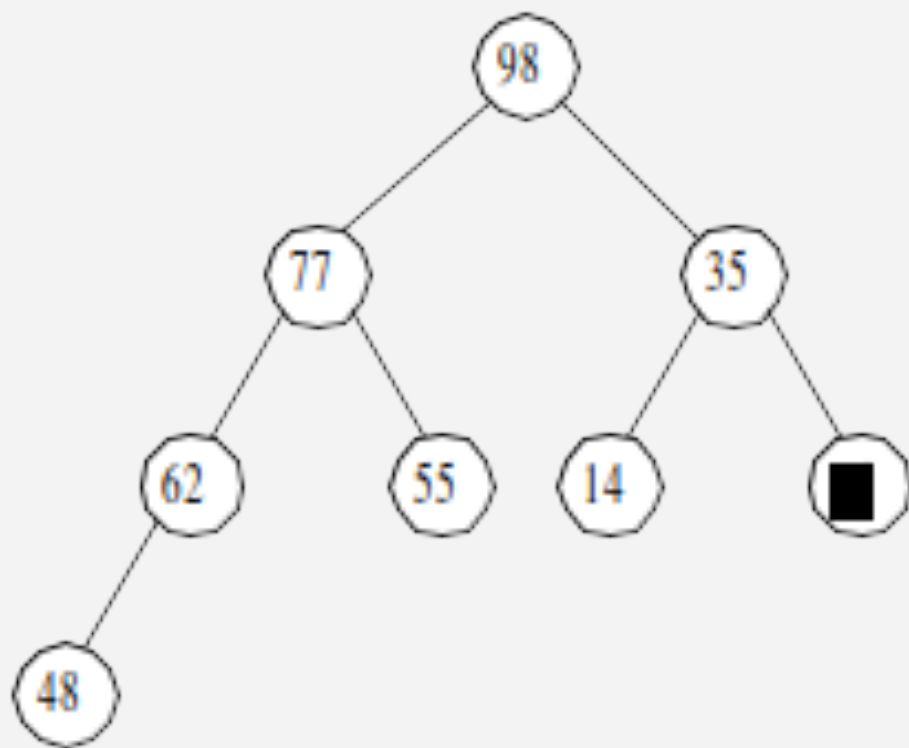
(大顶堆)

9.4.2 堆排序

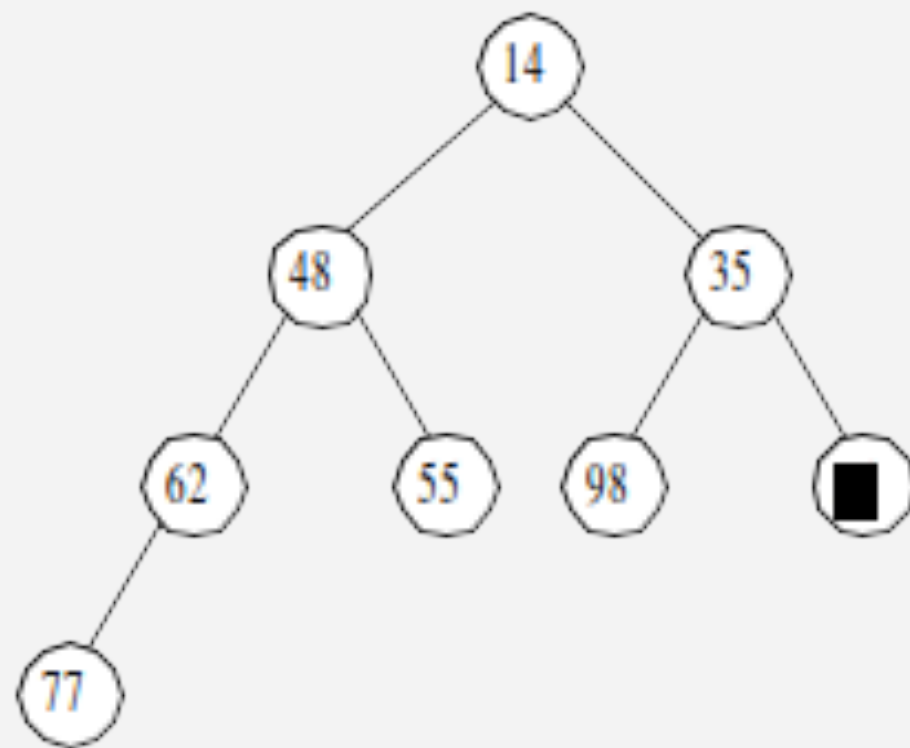
- 若将该数列视作完全二叉树，则 e_{2i+1} 是 e_i 的左孩子； e_{2i+2} 是 e_i 的右孩子
- 若一棵完全二叉树是堆，则树中的每个非终端结点的值均不大于（或不小于）其左、右孩子结点的值。根结点一定是这 n 个结点中的最小者或最大者。



9.4.2 堆排序

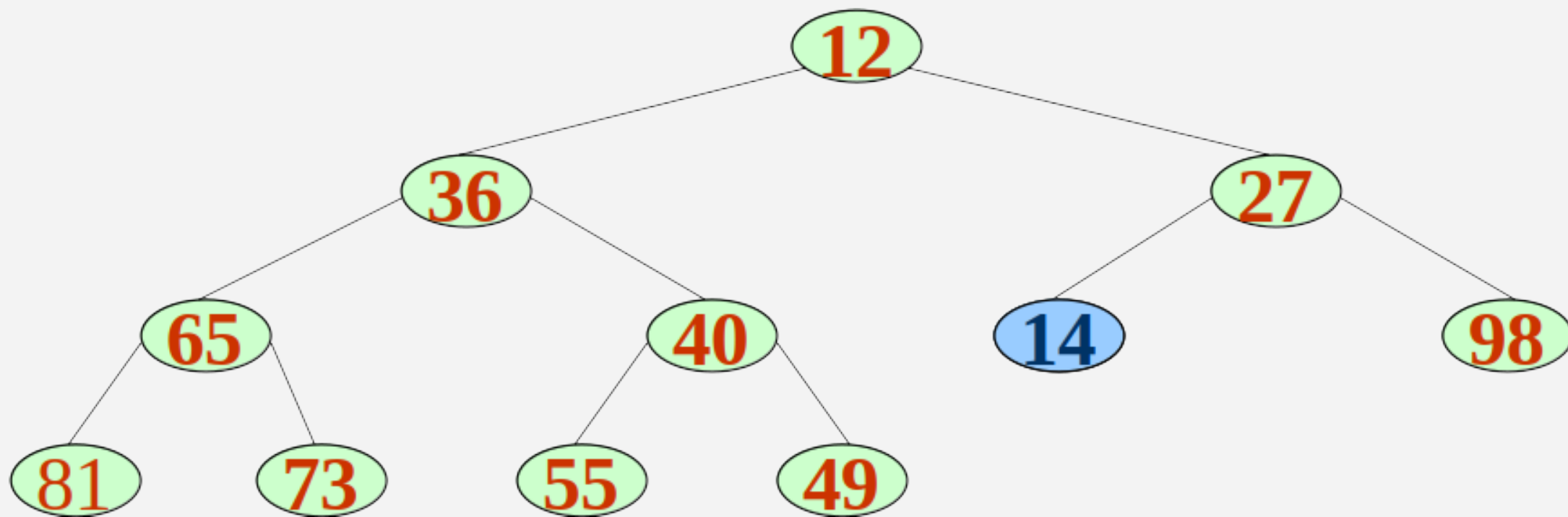


(a) 一个大根堆



(b) 一个小根堆

9.4.2 堆排序



不是堆

9.4.2 堆排序

堆排序的过程主要需要解决两个问题：

- 1、按堆定义将无序序列构建成一个堆；
- 2、如何在输出堆顶元素之后，调整剩余元素成为一个新的堆。

9.4.2 堆排序

问题1：如何由一个任意序列建初堆？

- 一个任意序列看成是对应的完全二叉树，由于叶子结点可以视为单元素的堆，因而可以反复利用“筛选”法，自底向上逐层把所有以非叶子结点为根的子树调整为堆，直到将整个完全二叉树调整为堆。
- 可以证明，最后一个非叶子结点位于 $\lfloor n/2 \rfloor$ 个元素， n 为二叉树结点数。因此，“筛选”须从第 $\lfloor n/2 \rfloor$ 个元素开始，逐层向上倒退，直到根结点。

9.4.2 堆排序

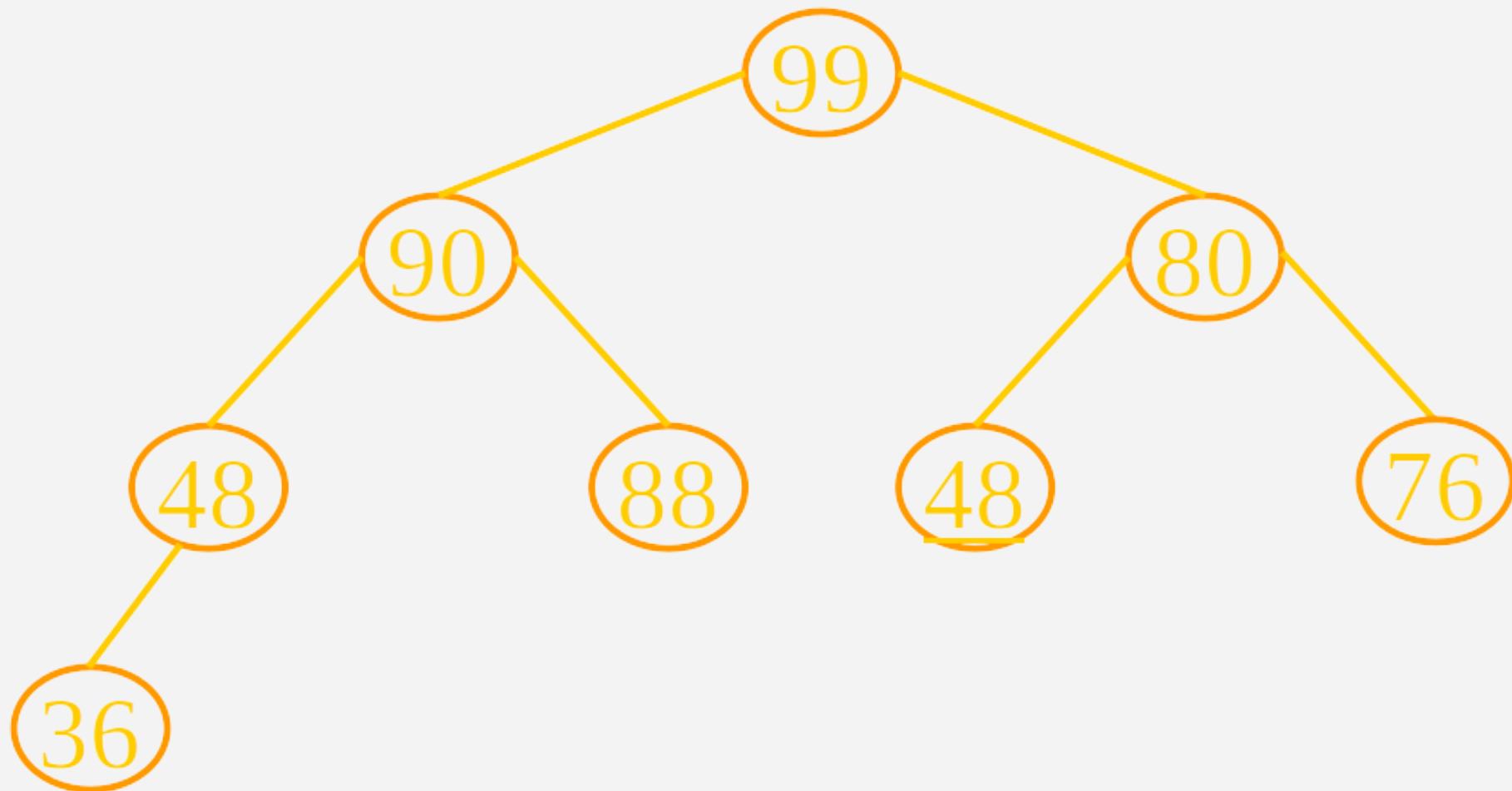
问题2：当堆顶元素改变时，如何重建堆？

- 假设输出堆顶元素之后，以堆中最后一个元素替代，此时根结点的左右子树均为堆，则仅需自上至下进行调整即可。首先以堆顶元素和其左右子树根结点比较，进行相应调整，观察调整之后破坏了哪一个堆，则需要对这个堆进行上述相同的调整，直至叶子结点。

- 称这个自堆顶至叶子的调整过程为“筛选”。

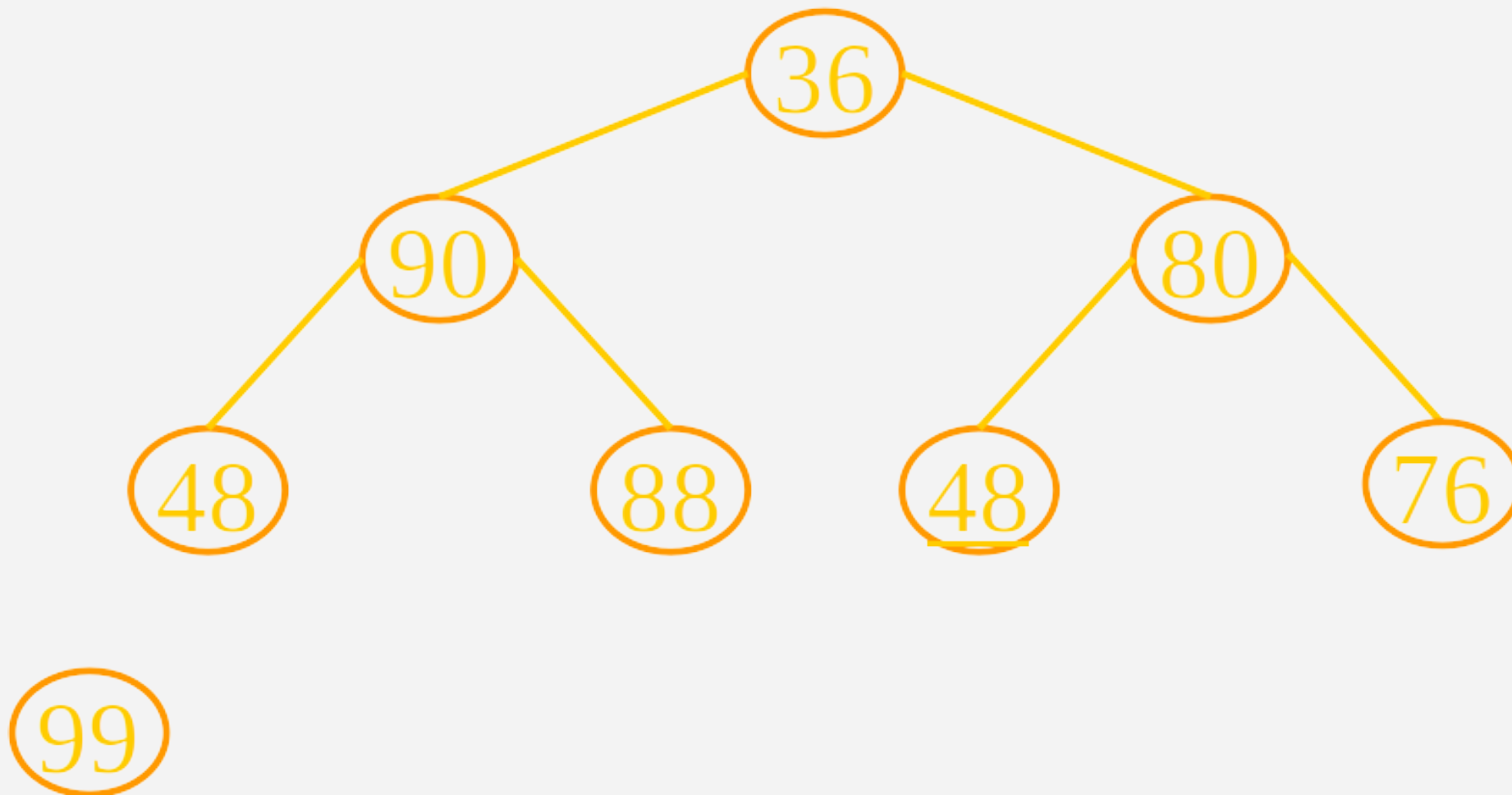
9.4.2 堆排序

堆顶元素改变时，如何重建堆



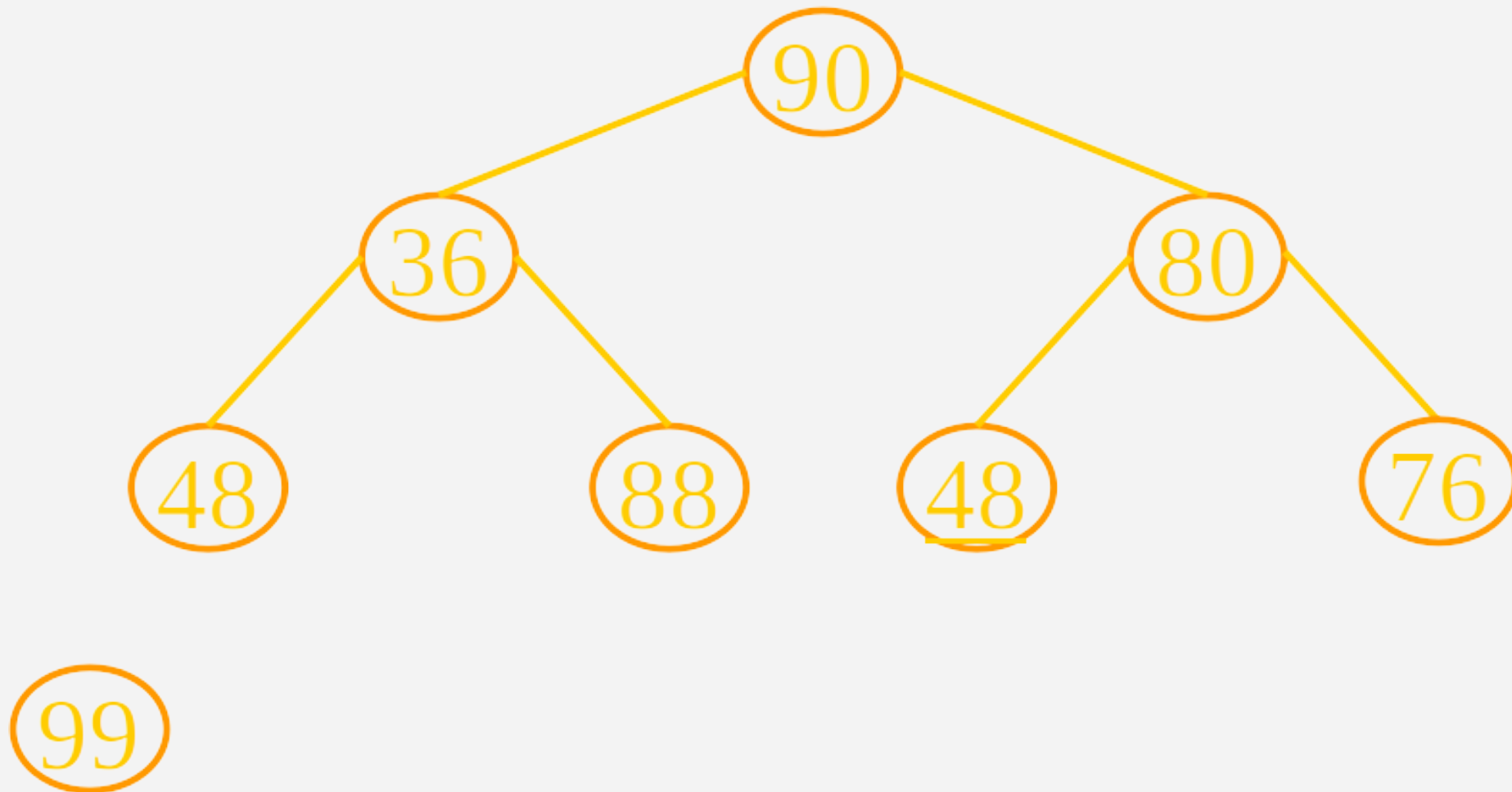
9.4.2 堆排序

堆顶元素改变时，如何重建堆



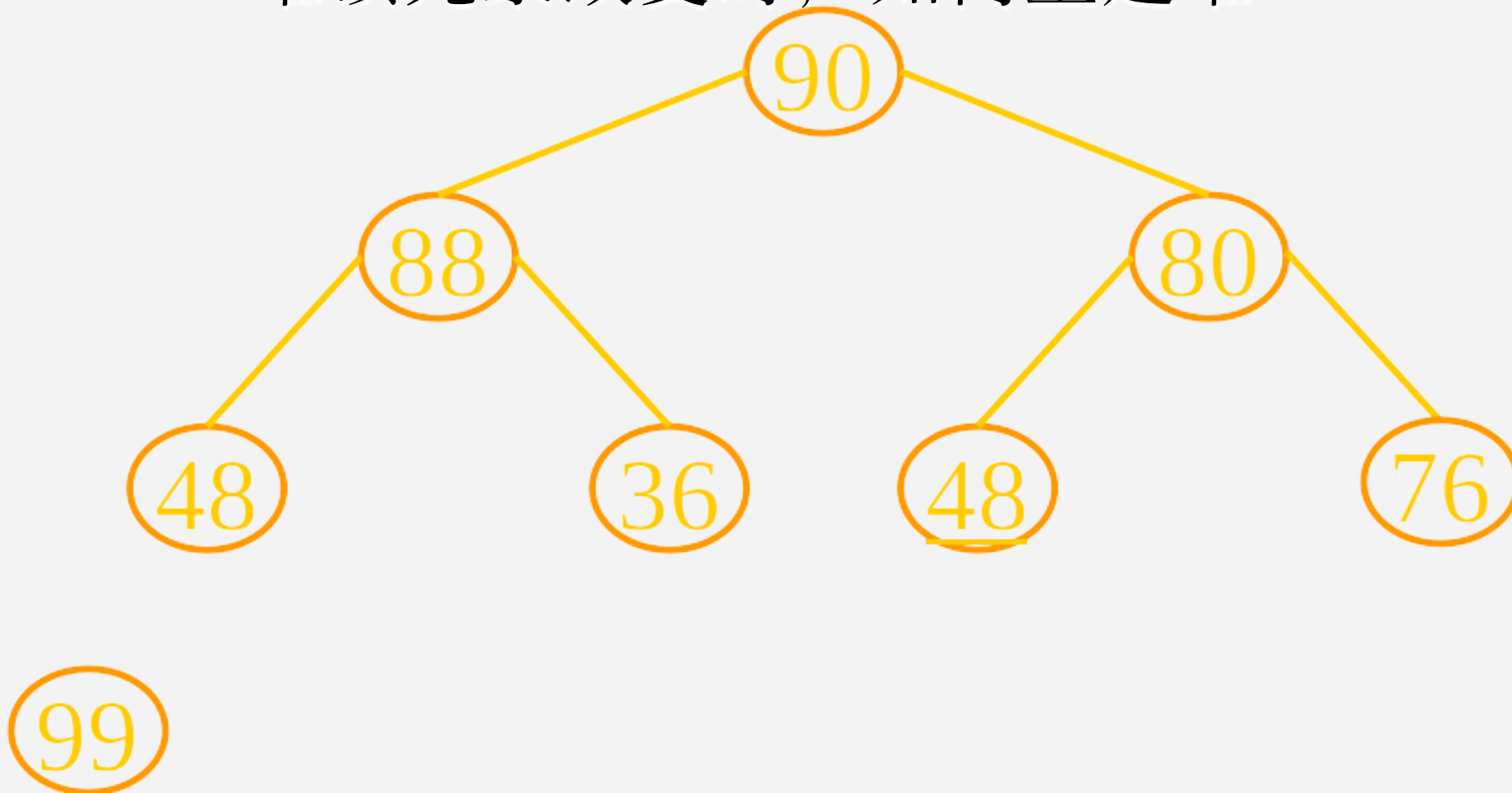
9.4.2 堆排序

堆顶元素改变时，如何重建堆



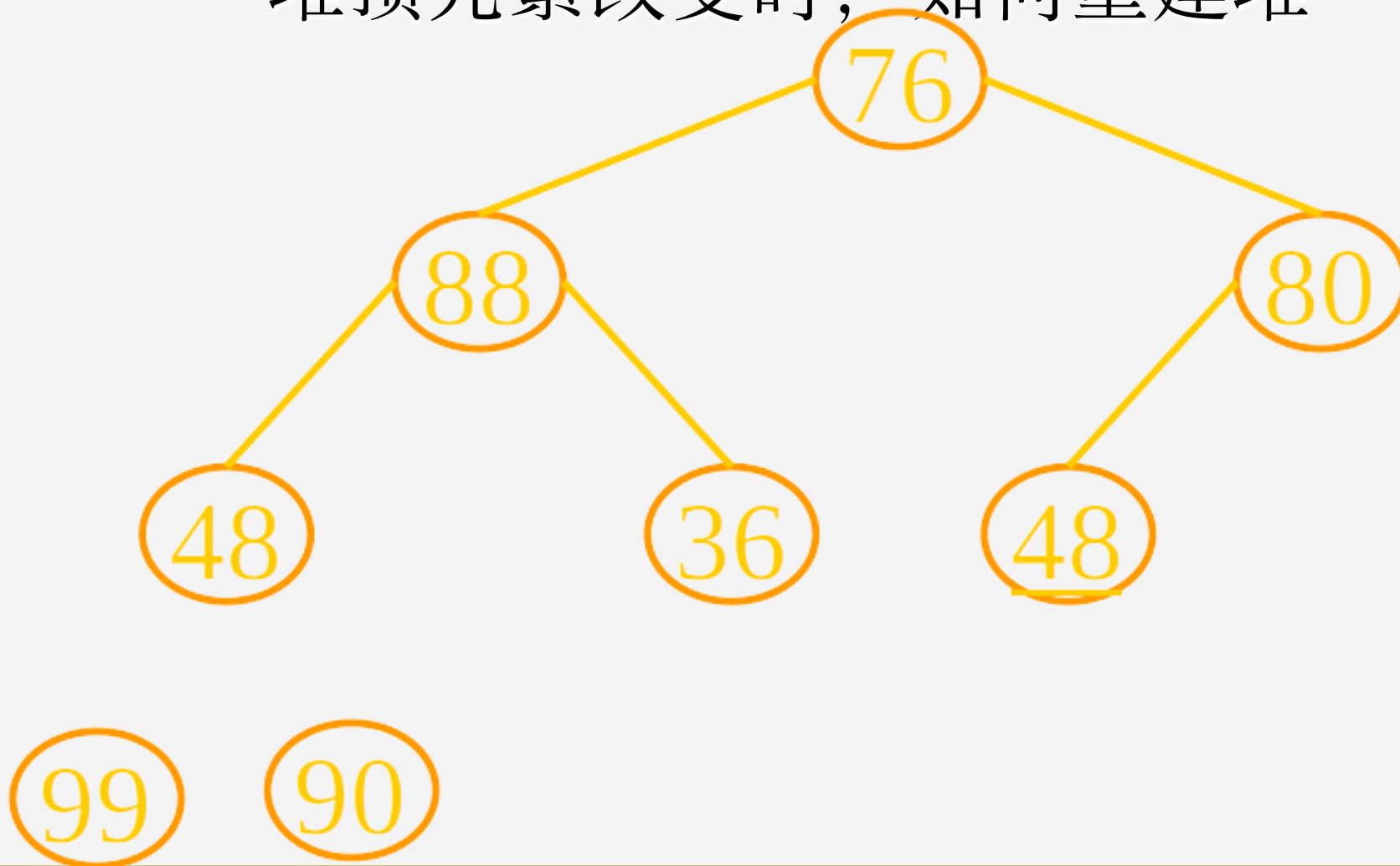
9.4.2 堆排序

堆顶元素改变时，如何重建堆



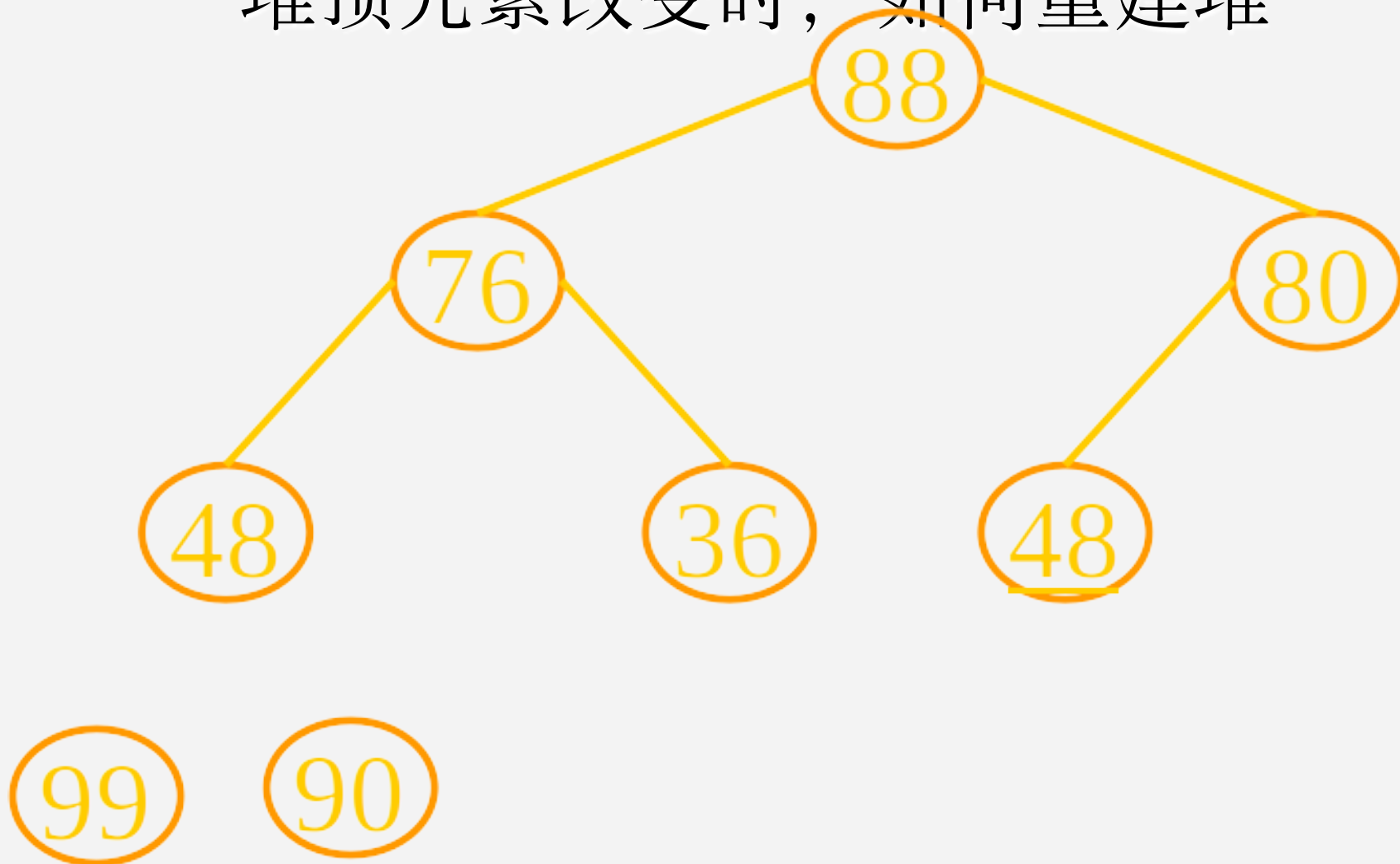
9.4.2 堆排序

堆顶元素改变时，如何重建堆



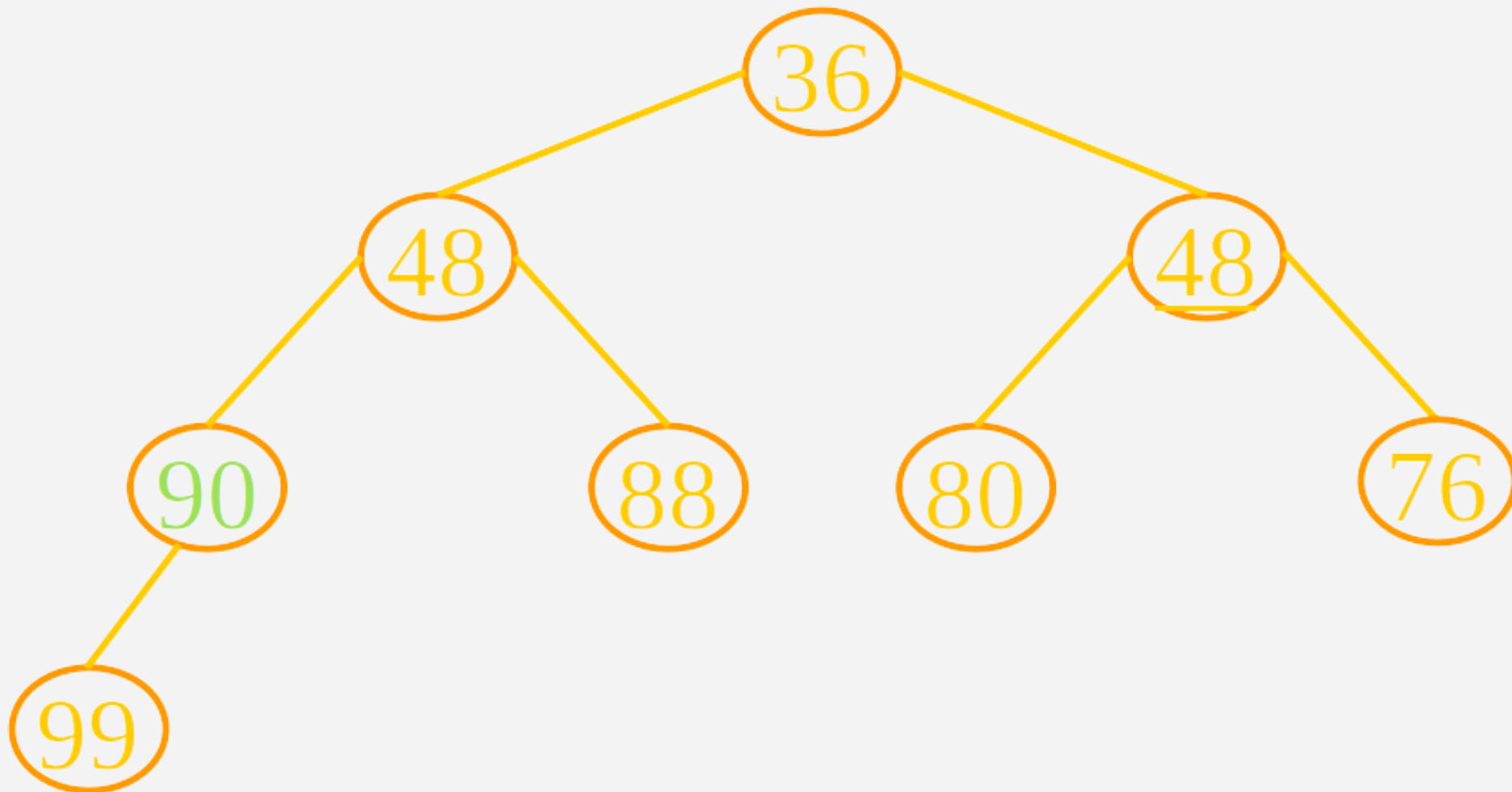
9.4.2 堆排序

堆顶元素改变时，如何重建堆



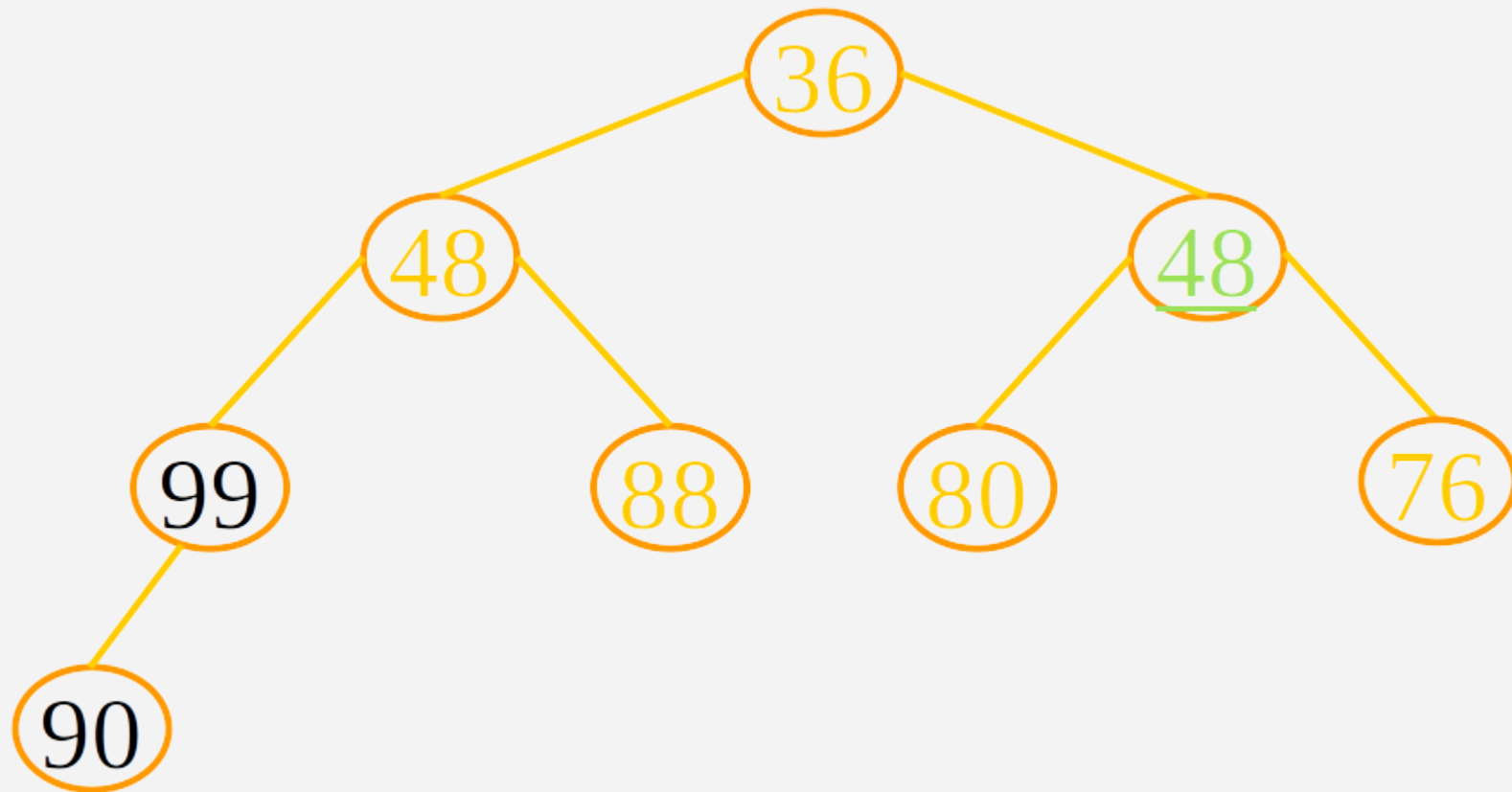
9.4.2 堆排序

无序序列建立初始堆



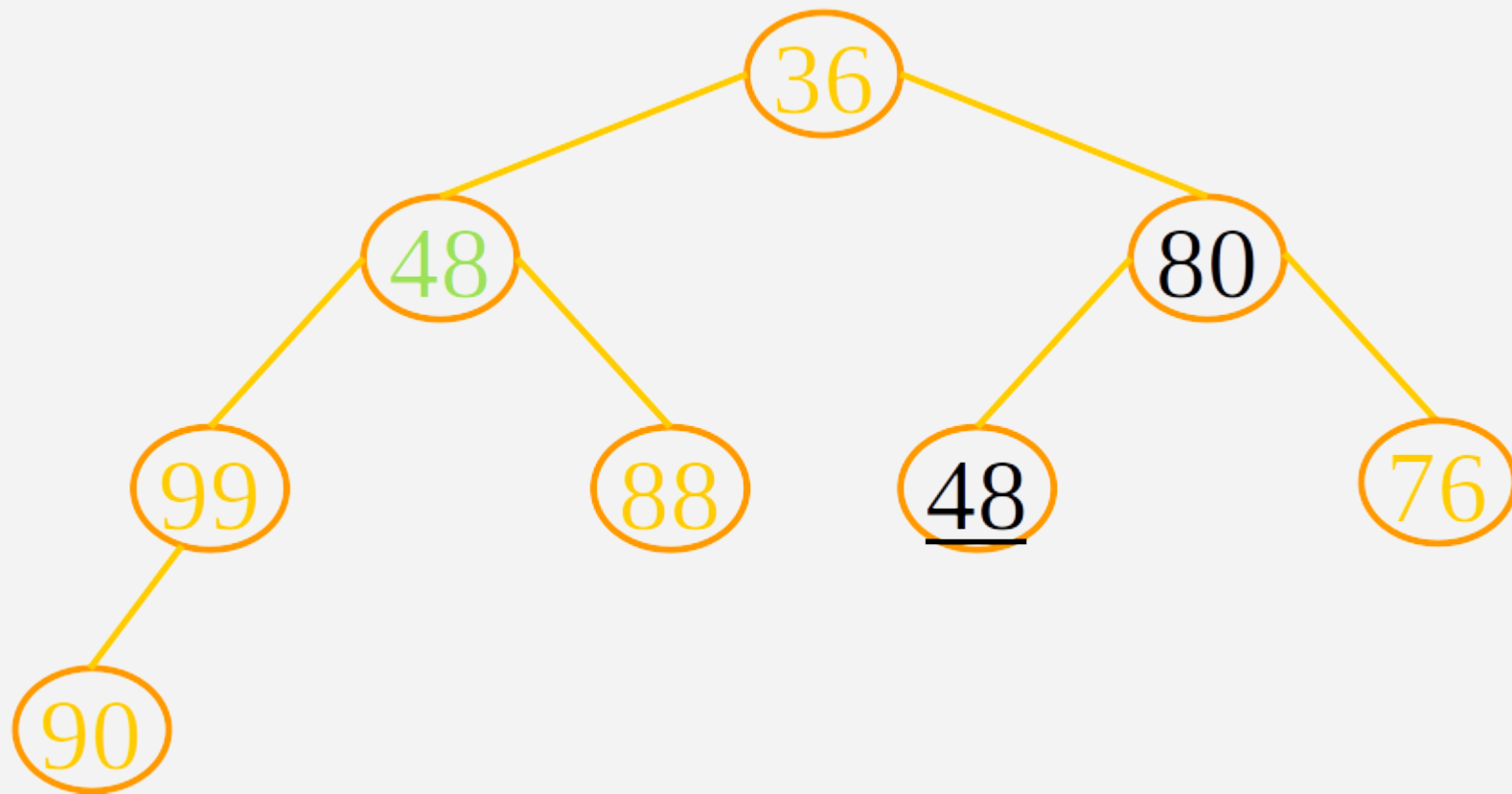
9.4.2 堆排序

无序序列建立初始堆



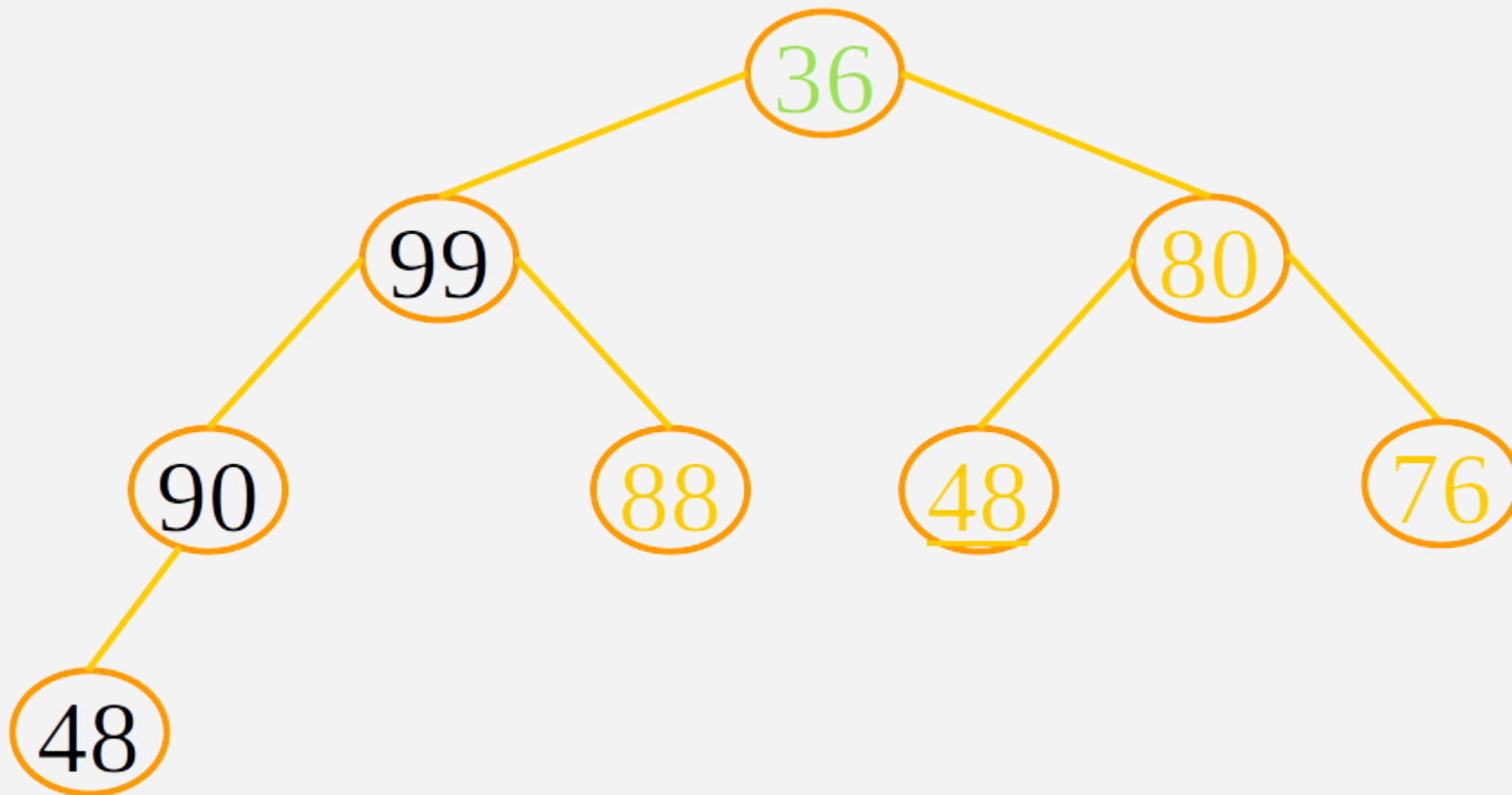
9.4.2 堆排序

无序序列建立初始堆



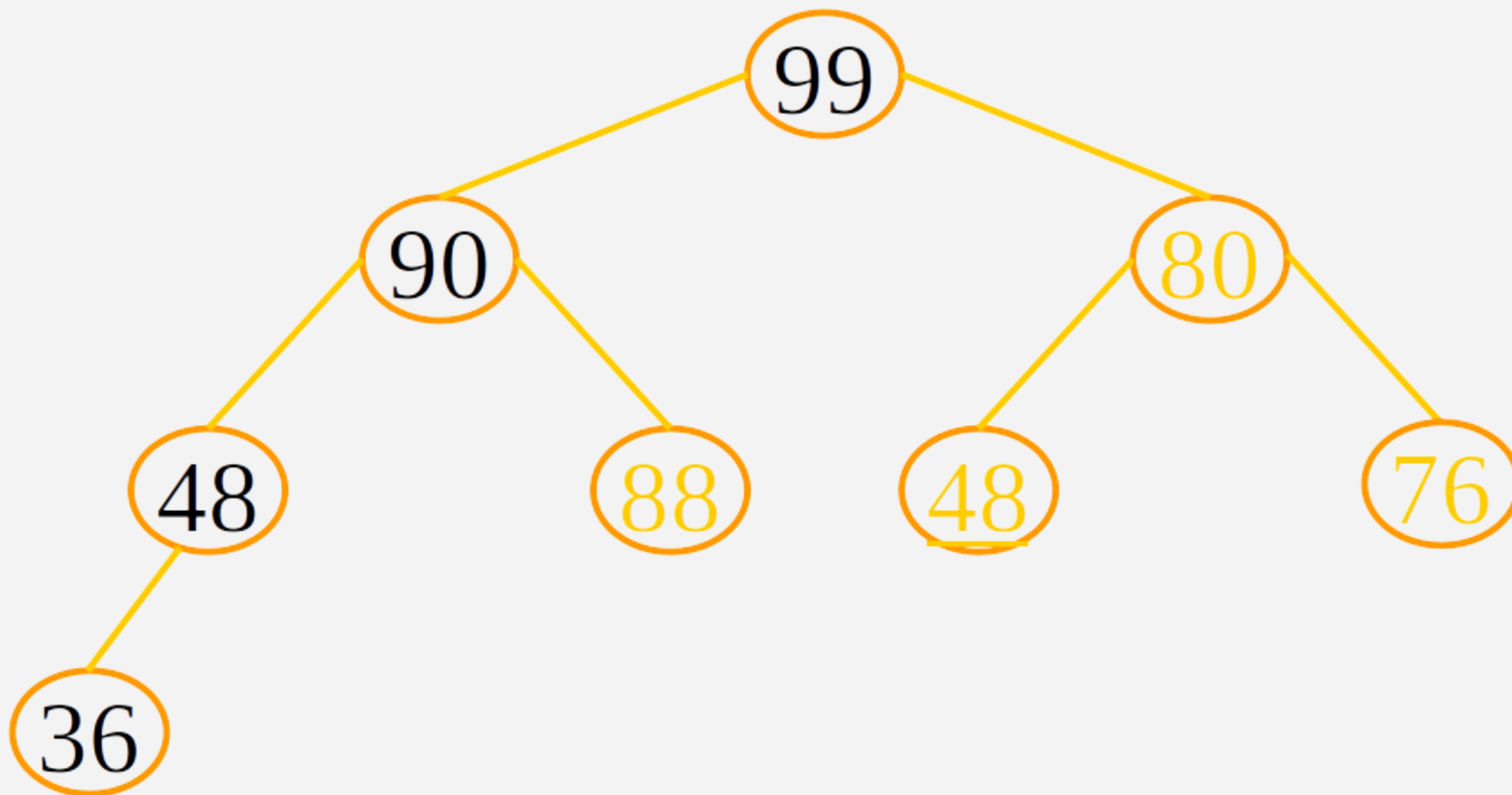
9.4.2 堆排序

无序序列建立初始堆



9.4.2 堆排序

无序序列建立初始堆



9.4.2 堆排序

如何利用堆进行排序？

- ① 将待排序记录按照堆的定义建初堆，并输出堆顶元素；
- ② 调整剩余的记录序列，利用筛选法将前 $n-i$ 个元素重新筛选并建成为一个新堆，再输出堆顶元素；
- ③ 重复执行步骤②，进行 $n-1$ 次筛选。新筛选成的堆会越来越小，而新堆后面的有序关键字会越来越多，最后使待排序记录序列成为一个有序的序列，这个过程称之为堆排序。

9.4.2 堆排序

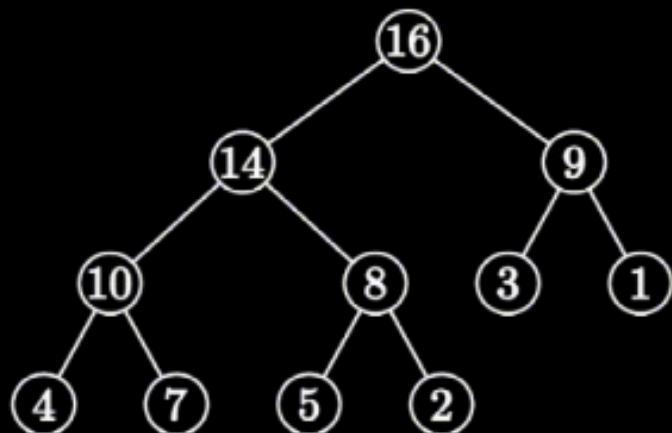
- 堆排序在最坏情况下，其时间复杂度也为 $O(n \lg n)$ ，这是堆排序的最大优点。
- 排序中只需要存放一个记录的辅助空间，因此也将堆排序称作原地排序。它不适用于待排序记录个数 n 较少的情况，但对于 n 较大的文件还是很有效的。

9.4.2 堆排序

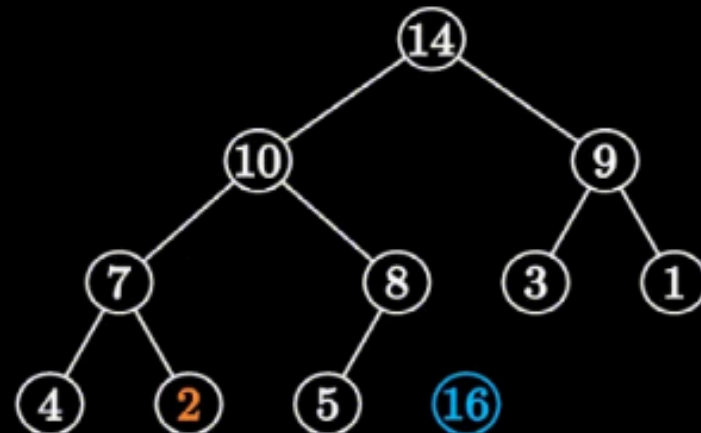
- 堆的物理结构，或者说存储结构，是一个数组，它使用顺序结构也就是数组来储存数据。而完全二叉树更适合使用顺序结构存储。
- 升序排序需要建立大顶堆；降序排序需要建立小顶堆。
- 实际上每一趟堆排序是堆顶节点与最后一个节点交换，即数组首元素 a_0 与 a_{n-i} 交换，再向下调整成为新的堆。

9.4.2 堆排序

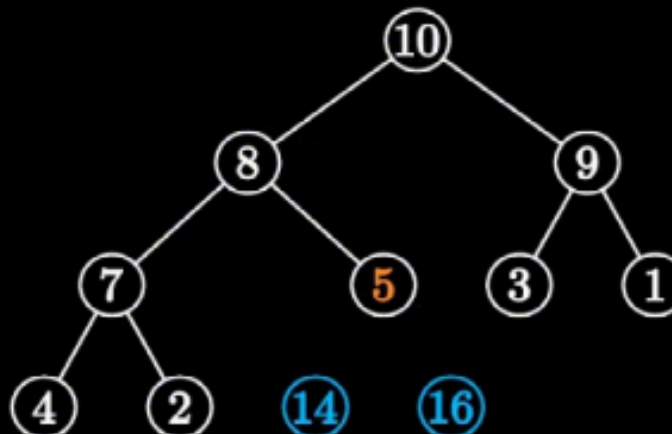
16 14 9 10 8 3 1 4 7 5 2



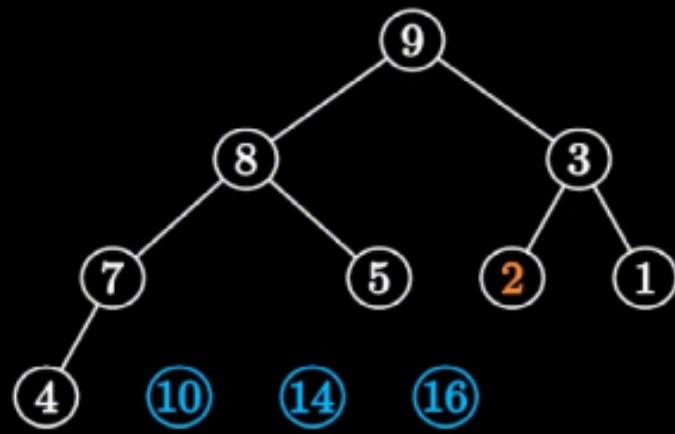
14 10 9 7 8 3 1 4 2 5 16



10 8 9 7 5 3 1 4 2 14 16



9 8 3 7 5 2 1 4 10 14 16



9.4.2 堆排序

堆的向下调整和向上调整

- 向下调整：初始建立堆；堆建立完毕，进入循环：将堆顶元素（数组的第一个元素）与堆的最后一个元素（数组的最后一个元素）交换，然后重新向下调整，但每次调整的范围都减小一个（即排除掉数组最后一个元素）。
- 向上调整：当向堆中插入新元素时，将元素插入堆的末尾，即数组的末尾。如破坏堆的性质，需要对该元素进行向上调整。向上调整法就是从新插入的节点开始，通过与其父节点的比较和交换，确保该节点的值不大于（对于大根堆）或不小于（对于小根堆）其父节点的值。时间复杂度为 $O(\log n)$ 。

9.4.2 堆排序

堆排序的TOP-K问题

- TOP-K问题是指在一个大数据集中找到前K个最大或最小的元素。这个问题在多个领域都非常常见，比如排名、选举、统计和游戏等。常见的例子包括找到考试成绩中的前10名、世界500强企业或者游戏中最活跃的100名玩家等等。
- 当数据量非常大时，一般的排序方法可能会因为数据量超过内存限制而变得不可行。此外，完整的排序操作的时间复杂度为 $O(n \lg n)$ ，这在数据量极大时效率低下。

9.4.2 堆排序

堆排序的TOP-K问题

•堆排序提供了一个更为高效的解决方案，时间复杂度为 $O(n \log K)$ ，这对于大数据来说是一个巨大提升。基本思路是：

- 用数据集合中前K个元素来建堆。如果我们需要找到前K个最大的元素，则建立一个小顶堆；如果我们需要找到前K个最小的元素，则建立一个大顶堆。
- 用剩余的 $N-K$ 个元素依次与堆顶元素比较。
- 如果当前元素比堆顶元素大（在寻找最大元素时）或小（在寻找最小元素时），则将其与堆顶元素替换，并重新调整堆。
- 提取堆中的元素。经过上述过程后，堆中剩余的K个元素就是我们要找的前K个最大或最小的元素。

```
// 调整以 index 为根的子树为最大堆 (heapify)
void heapify(int arr[], int n, int index) {
    int largest = index;    // 初始化最大值为根
    int left = 2 * index + 1; // 左子节点
    int right = 2 * index + 2; // 右子节点

    // 如果左子节点存在且大于根
    if (left < n && arr[left] > arr[largest])
        largest = left;

    // 如果右子节点存在且大于当前最大值
    if (right < n && arr[right] > arr[largest])
        largest = right;

    // 如果最大值不是根，则交换并继续调整
    if (largest != index) {
        int temp = arr[index];
        arr[index] = arr[largest];
        arr[largest] = temp;

        // 递归调整受影响的子树
        heapify(arr, n, largest);
    }
}
```

```
// 堆排序主函数（升序）
void heapSort(int arr[], int n) {
    // 1. 构建最大堆（从最后一个非叶子节点开始）
    for (int i = n / 2 - 1; i >= 0; i--) {
        heapify(arr, n, i);
    }

    // 2. 逐个将堆顶（最大值）移到末尾，并重新调整堆
    for (int i = n - 1; i > 0; i--) {
        // 将当前最大值（堆顶）与末尾元素交换
        int temp = arr[0];
        arr[0] = arr[i];
        arr[i] = temp;

        // 对剩余元素（0 到 i-1）重新调整为最大堆
        heapify(arr, i, 0);
    }
}
```

9.4.2 堆排序

已知关键字序列5, 8, 12, 19, 28, 20, 15, 22是小根堆（最小堆），插入关键字3，调整后得到的小根堆是（ A ）

- A. 3, 5, 12, 8, 28, 20, 15, 22, 19
- B. 3, 5, 12, 19, 20, 15, 22, 8, 28
- C. 3, 8, 12, 5, 20, 15, 22, 28, 19
- D. 3, 12, 5, 8, 28, 20, 15, 22, 19

9.5 归并排序

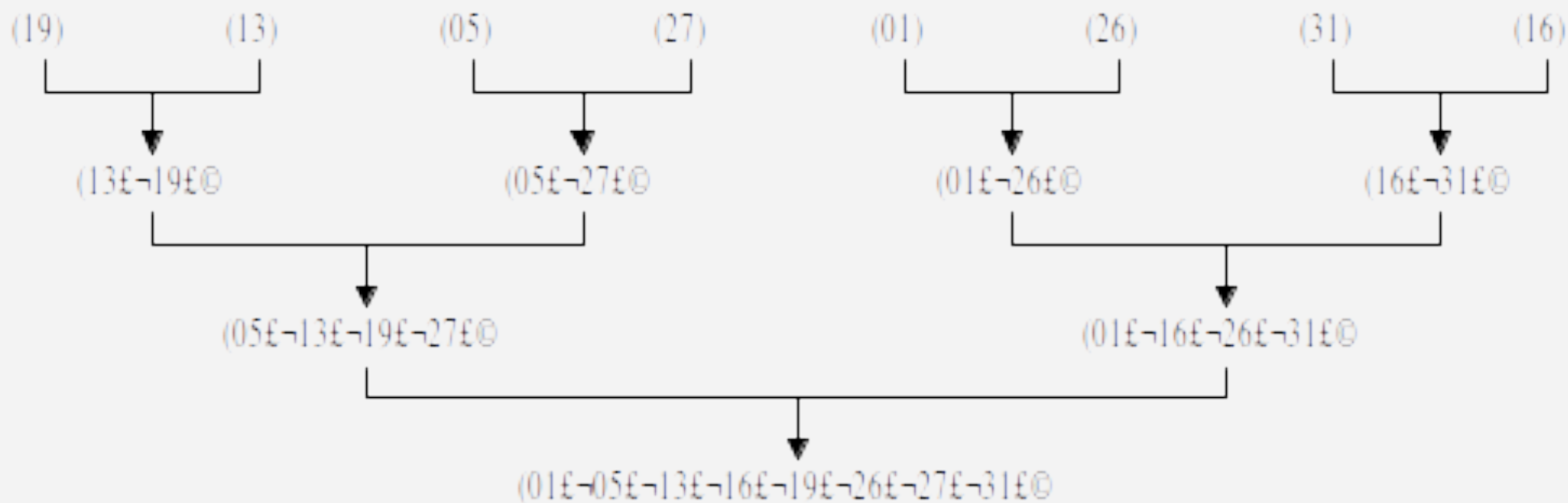
- 归并排序是一种另类排序方法。所谓归并是指将两个或两个以上的有序序列合并成一个新的有序序列。
- 归并排序的基本思想是将一个具有 n 个待排序记录的序列看成是 n 个长度为1的有序序列，然后进行两两归并，得到 $n/2$ 个长度为2的有序序列，再进行两两归并，得到 $n/4$ 个长度为4的有序序列，如此重复，直至得到一个长度为 n 的有序序列为止。

9.5 归并排序

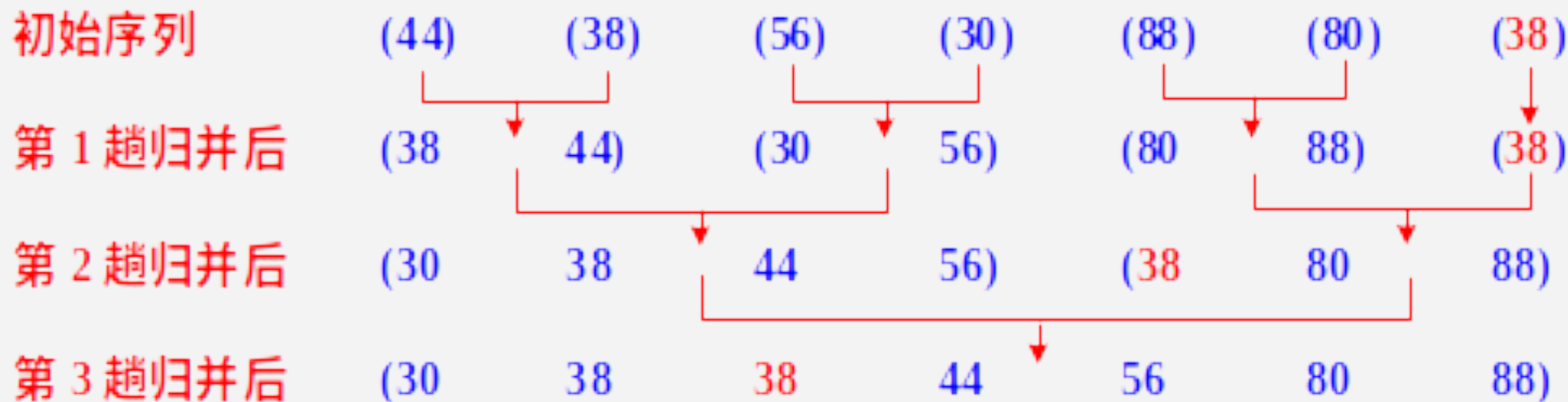
- 如果每次将3个有序序列合并为一个新的有序序列，则称为3路归并排序，以此类推。
- 对于内部排序来讲，2路归并排序就能完全满足需要。

9.5 归并排序

- 2路归并排序法的基本操作是将待排序列中相邻的两个有序子序列合并成一个有序序列。（N个元素的序列一分为二。注意程序实现两个子序列的合并算法。）



9.5 归并排序



9.5 归并排序

- 时间复杂度也为 $O(n \lg n)$ ，空间复杂度为 $O(n)$ 。
- 与快速排序和堆排序相比，归并排序的最大特点是它是一种稳定的排序方法。
- 归并排序时，对于已经有序的两个子序列，其比较次数会大大降低。
- 注意归并排序的手动操作方法。

9.5 归并排序

- 类似2路归并排序，可设计多路归并排序法，归并的思想主要用于外部排序。
- 外部排序可分两步：① 待排序记录分批读入内存，用某种方法在内存排序，组成有序的子文件，再按某种策略存入外存；② 子文件多路归并，形成较长的有序子文件，再存入外存，如此反复，直到整个待排序文件有序。

9.6 基数排序

- 基数排序是一种借助“多关键字排序”的思想来实现“单关键字排序”的内部排序算法。
- 多关键字的排序和链式基数排序。

9.6.1 多关键字排序

n 个元素的序列 $\{e_1, e_2, \dots, e_n\}$ 对关键字 $(k_i^0, k_i^1, \dots, k_i^{d-1})$ 有序是指:

- 对于序列中任意两个元素 e_i 和 e_j ($1 \leq i < j \leq n$) 都满足下列(词典)有序关系: $(k_i^0, k_i^1, \dots, k_i^{d-1}) < (k_j^0, k_j^1, \dots, k_j^{d-1})$
- 其中: k^0 被称为 “最高” 位关键字; k^{d-1} 被称为 “最低” 位关键字

9.6.1 多关键字排序

实现多关键字排序通常有两种作法：

- 最高位优先MSD法
- 最低位优先LSD法（最常见的方法）

9.6.1 多关键字排序

- 最高位优先MSD法

先对 k^0 进行排序，并按 k^0 的不同值将元素序列分成若干子序列之后，分别对 k^1 进行排序， $\dots\dots$ ，依次类推，直至最后对最次位关键字排序完成为止。

- 最低位优先LSD法

先对 k^{d-1} 进行排序，然后对 k^{d-2} 进行排序，依次类推，直至对最主位关键字 k^0 排序完成为止。

9.6.1 多关键字排序

- 例如：学生记录含三个关键字：系别、班号和班内的序列号，其中以系别为最高位关键字。LSD的排序过程如下

无序序列	3,2,30	1,2,15	3,1,20	2,3,18	2,1,20
对 K^2 排序	1,2, 15	2,3, 18	3,1, 20	2,1, 20	3,2, 30
对 K^1 排序	3, 1 ,20	2, 1 ,20	1, 2 ,15	3, 2 ,30	2, 3 ,18
对 K^0 排序	1 ,2,15	2 ,1,20	2 ,3,18	3 ,1,20	3 ,2,30

9.6.1 多关键字排序

- 使用LSD法进行排序时，对每个关键字可以采用“分配-收集”的方法，其好处是不需要进行关键字间的比较。
- 对于数字型或字符型的单关键字，可以看成是由多个数位或多个字符构成的多关键字，此时可以采用这种“分配-收集”的办法进行排序。

9.6.2 链式基数排序

- 基数排序是借助于多关键字排序思想进行排序的一种排序方法。
- 该方法将排序关键字 K 看作是由多个关键字组成的组合关键字，即 $K=k^1k^2\cdots k^d$ 。每个关键字 k^i 表示关键字的一位，其中 k^1 为最高位， k^d 为最低位， d 为关键字的位数。
- 在实现时可将数据按关键字分配到线性链表中，然后再对所得的线性链表进行收集。

9.6.2 链式基数排序

- 例如，对于关键字序列（101，203，567，231，478，352），可以将每个关键字K看成由三个单关键字组成，即 $K = k_1k_2k_3$ ，每个关键字的取值范围为 $0 \leq k_i \leq 9$ ，所以每个关键字可取值的数目为10。

9.6.2 链式基数排序

- 通常将关键字取值的数目称为基数，用符号 r 表示，在这个例子中 $r=10$ 。
- 对于关键字序列 (AB, BD, ED) 可以将每个关键字看成是由二个单字母关键字组成的复合关键字，并且每个关键字的取值范围为“A~Z”，所以关键字的基数 $r=26$ 。

9.6.2 链式基数排序

- 我们在这里讲述的基数排序是指用多关键字的LSD方法排序，即对待排序的记录序列按照复合关键字从低位到高位顺序交替地进行“分组”、“收集”，最终得到有序的记录序列。

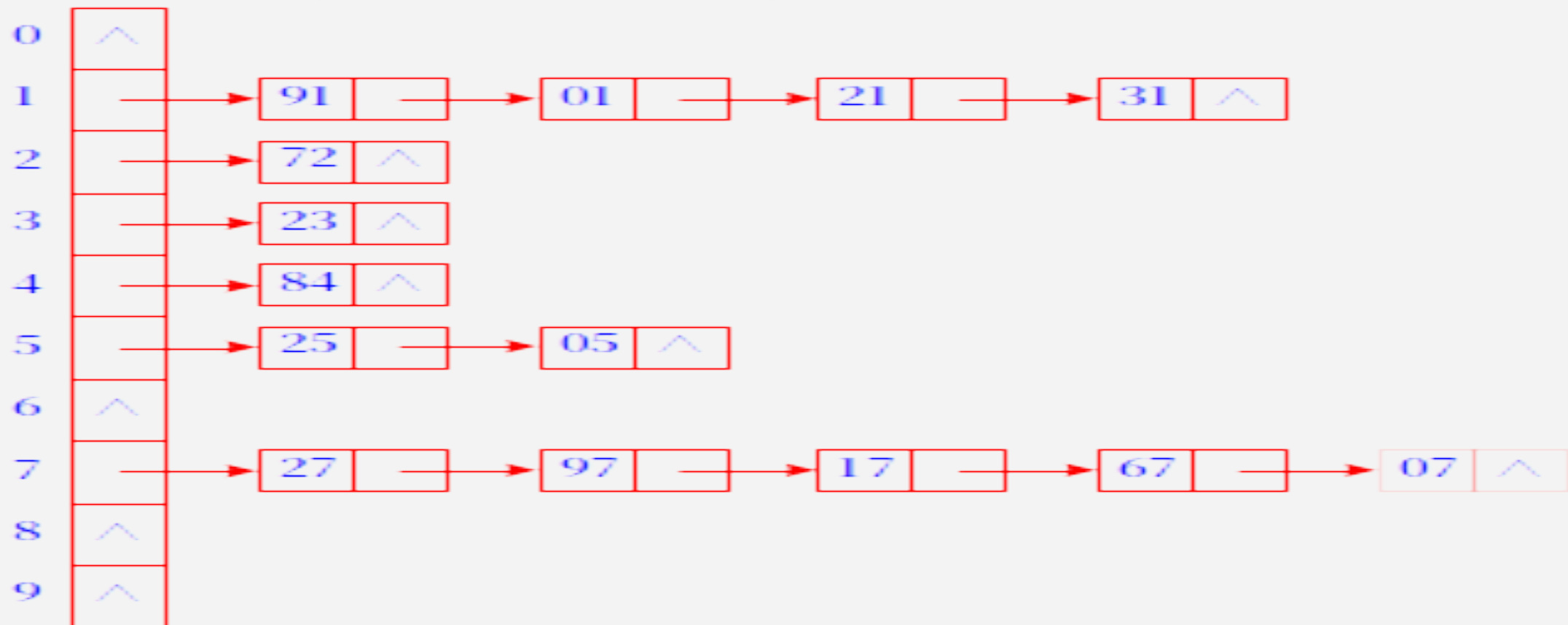
9.6.2 链式基数排序

- 在此我们将一次“分组”、“收集”称为一趟。对于由 d 位关键字组成的复合关键字，需要经过 d 趟的“分配”与“收集”。
- 基数排序的时间复杂度为 $O(d(n+r))$ 。其中分配为 $O(n)$ ，收集为 $O(r)$ (r 为“基”)， d 关键字的位数，也是“分配-收集”的趟数。
- 空间复杂度从 $O(rn)$ 改进为 $O(r+n)$ 。

9.6.2 链式基数排序

{27, 91, 01, 97, 17, 23, 72, 25, 05, 67, 84, 07, 21, 31}

第一趟分配(按个位数进行分配)

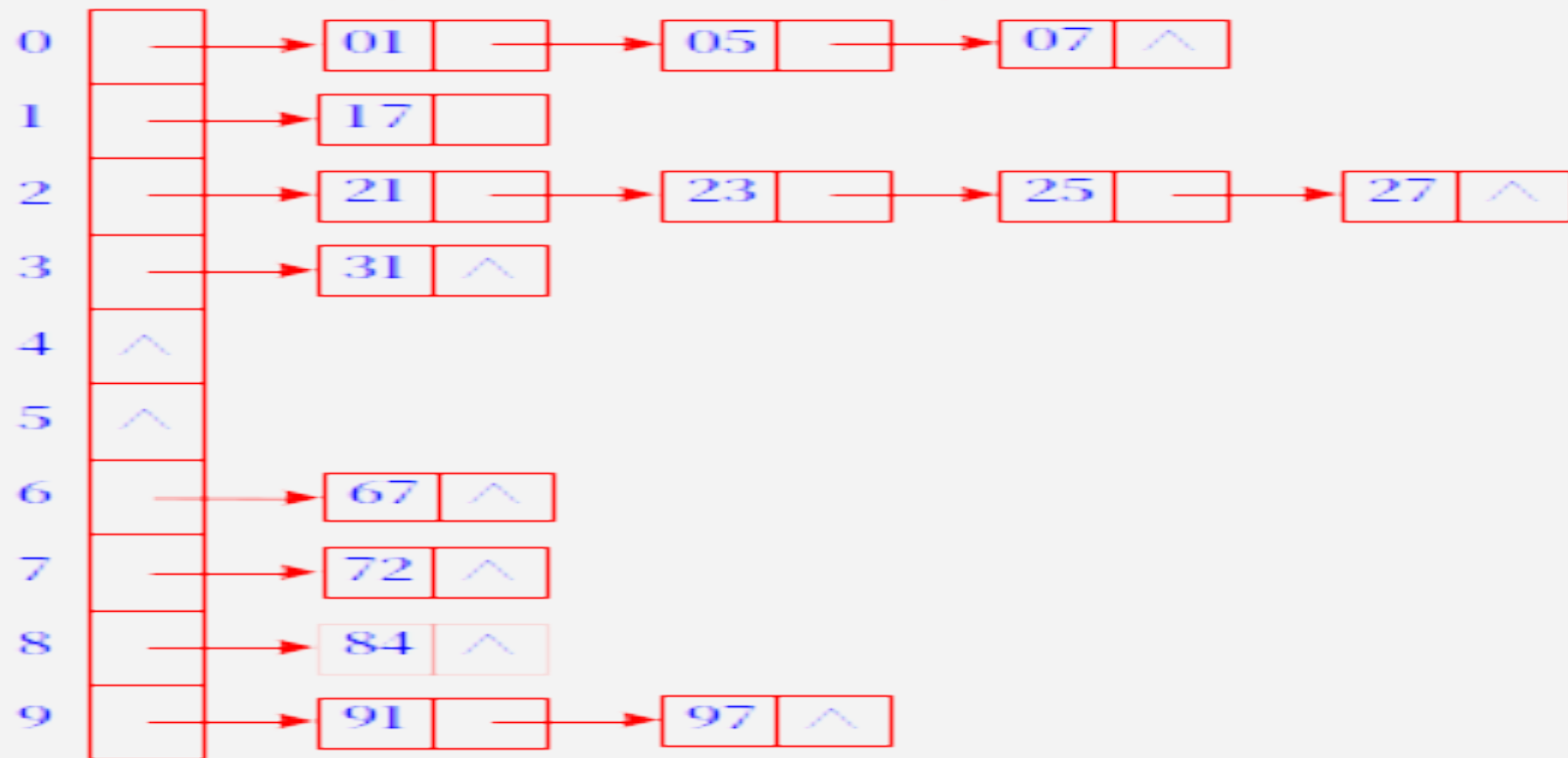


第一趟收集结果: {91, 01, 21, 31, 72, 23, 84, 25, 05, 27, 97, 17, 67, 07}

9.6.2 链式基数排序

{91, 01, 21, 31, 72, 23, 84, 25, 05, 27, 97, 17, 67, 07}

第二趟分配(按十位数进行分配)



第二趟收集结果: {{ 01, 05, 07, 17, 21, 23, 25, 27, 31, 67, 72, 84, 91, 97}}

9.7 各种内部排序方法讨论

平均的时间性能

- 时间复杂度为 $O(n \log_2 n)$: 快速排序、堆排序和归并排序
- 时间复杂度为 $O(n^2)$: 直接插入排序、起泡排序和简单选择排序
- 时间复杂度为 $O(dn)$: 基数排序

9.7 各种内部排序方法讨论

- 排序元素序列按关键字顺序有序
 - 直接插入排序能达到 $O(n)$ 的时间复杂度，快速排序的时间性能蜕化为 $O(n^2)$ 。
- 排序的时间性能不随关键字分布而改变的排序
 - 简单选择排序、起泡排序、堆排序和归并排序的时间性能不随元素序列中关键字的分布而改变。

9.7 各种内部排序方法讨论

- 排序方法的稳定性能
 - 稳定的排序方法指的是，对于两个关键字相等的元素，它们在序列中的相对位置，在排序之前和经过排序之后，没有改变。

排序前: { $e_i(k)$ $e_j(k)$ }

排序后: { $e_i(k)$ $e_j(k)$ }

例如：

排序前 (56, 34, 47, 23, 66, 18, 82, 47)

若排序后得到结果

(18, 23, 34, 47, 47, 56, 66, 82)

则称该排序方法是稳定的；

若排序后得到结果

(18, 23, 34, 47, 47, 56, 66, 82)

则称该排序方法是不稳定的。

9.7 各种内部排序方法讨论

- 对于不稳定的排序方法，只要能举出一个实例说明即可。例如：对{ 4, 3, 4, 2 }进行快速排序，得到{ 2, 3, 4, 4 }。
- 快速排序、所有选择排序（包括堆排序）和希尔排序是不稳定的排序方法。

9.8 外部排序

- 外存信息的存取：磁带信息的存取和磁盘信息的存取。
- 内部排序的特点是排序过程中所有数据都在内存中。
- 当要排序的数据元素非常多，内存中不能一次进行处理，只能将它们以文件的形式存放于外存中，排序时将一部分数据元素调入内存进行处理，在排序过程中不断地在内存和外存之间传送数据，这样的排序方法称为外部排序。

9.8 外部排序

外部排序的基本过程，由相对独立的两个步骤组成

- 按可用内存大小，利用内部排序方法，构造若干（元素的）有序子序列，通常称外存中这些元素有序子序列为“归并段”；
- 通过“归并”，逐步扩大（元素的）有序子序列的长度，直至外存中整个元素序列按关键字有序为止。

9.8 外部排序

例如：假设有一个含10,000个元素的磁盘文件，而当前所用的计算机一次只能对1000个元素进行内部排序，则首先利用内部排序的方法得到10个初始归并段，然后进行逐趟归并。假设进行2-路归并(即两两归并)，则

- 第一趟由10个归并段得到5个归并段；
- 第二趟由 5 个归并段得到3个归并段；
- 第三趟由 3 个归并段得到2个归并段；
- 最后一趟归并得到整个元素的有序序列。

9.8 外部排序

分析上述外排过程中访问外存(对外存进行读/写)的次数:

假设“数据块”的大小为200, 即每一次访问外存可以读/写200个元素。则对于10,000个元素, 处理一遍需访问外存100次(读和写各50次)。

- 1) 求得10个初始归并段需访问外存100次;
- 2) 每进行一趟归并需访问外存100次;
- 3) 总计访问外存 $100 + 4 \times 100 = 500$ 次。

9.8 外部排序

- 外排总的还应包括内部排序所需时间和逐趟归并时进行内部归并的时间，显然，除去内部排序的因素外，外部排序的时间取决于逐趟归并所需进行的“趟数”。
- 例如，若对上述例子采用5-路归并，则只需进行2趟归并，总的访问外存的次数将压缩到 $100 + 2 \times 100 = 300$ 次。

9.8 外部排序

- 一般情况下，假设待排元素序列含 m 个初始归并段，外排时采用 k -路归并，则归并趟数为 $\lceil \log_k m \rceil$ 。
- 随 k 的增大归并的趟数将减少，因此对外排而言，通常采用多路归并。 k 的大小可选，但需综合考虑各种因素。

若数据元素序列11, 12, 13, 7, 8, 9, 23, 4, 5是采用下列排序方法之一得到的第二趟排序后的结果, 则该排序算法只能是 (B)

- A. 起泡排序
- B. 插入排序
- C. 选择排序
- D. 二路归并排序

数据表中有10000个元素, 如果仅需求出其中最大的10个元素, 则采用 (C) 排序算法最节省时间。

- A) 快速排序
- B) 希尔排序
- C) 堆排序
- D) 直接选择排序

【分析】本题中堆排序与直接选择排序每一趟都可选择出一个当前元素中的最大者, 显然采用堆排序最节省时间。

每一趟都能选出一个元素放在其最终位置上，并且不稳定的排序算法是（ B ）。

- A) 冒泡排序 B) 简单选择排序
C) 希尔排序 D) 直接插入排序

【分析】所有选择排序都是不稳定的排序方法，对于简单选择排序每一趟都能选出一个元素放在其最终位置上，所以本题应选择B。

已知序列25，13，10，12，9是大根堆，在序列尾部插入新元素18，将其再调整为大根堆，调整过程中元素之间进行的比较次数是（ B ）。

- A) 1 B) 2 C) 4 D) 5